GumSwap: Griefing-Free Universal Multi-Party Atomic Swaps

Dongkun Hou[†], Yuanzhe Zhang[§], Shujie Cui[†], Tsz Hon Yuen[†], Joseph K. Liu[†], and Jiangshan Yu[‡]
[†]Monash University, Melbourne, Australia
{dongkun.hou, shujie.cui, john.tszhonyuen, joseph.liu}@monash.edu
[§]Nanyang Technological University, Singapore
yuanzhe.zhang@ntu.edu.sg
[‡]The University of Sydney, Sydney, Australia
jiangshan.yu@sydney.edu.au

***Abstract*—Universal multi-party swaps were proposed for secure cross-chain cryptocurrency exchanges across multiple blockchains that require only signature verification from the underlying blockchains. However, existing universal swap protocols remain vulnerable to griefing attacks, where a deviating party aborts the swap to lock a compliant party's assets for a long period, potentially causing indirect economic losses. A natural approach is to lift existing griefing-free solutions to the universal setting; however, we observe that this direct approach still faces three key challenges: (i) a *timeout race attack*, which arises from the absence of an upper bound on the transaction validity; (ii) a *premium escape attack*, which results from multiple refund transactions for the same assets being simultaneously valid; and (iii) a *topological limitation*, which implies that universal multi-party swaps can support only a special class of strongly connected digraphs, called *reuniclus graphs*.**

In this paper, we propose GumSwap, a Griefing-free universal multi-party atomic Swap, which guarantees that a compliant party receives a premium if its asset is locked but not redeemed. To mitigate the timeout race attack and the premium escape attack, we impose minimum timeout intervals for the principal and premium timeouts, respectively, and introduce an *asset migration mechanism* that ensures that, during any time interval, at most one refund transaction is valid. Given the topological limitations of universal swap protocols, we further design a novel premium distribution mechanism that accommodates two classes of leaders in reuniclus graphs. Our experimental results demonstrate that GumSwap can be performed in less than 0.5 seconds per party, while reducing gas costs by $10.3\times$ compared with existing contract-based solutions.

***Index Terms*—Atomic Swaps, Universality, Griefing Attacks, Reuniclus Graphs**

I. Introduction

Atomic swaps are crucial protocols for blockchain interoperability, enabling secure and trustless asset exchange across distinct blockchains [1], [2]. Universal atomic swaps [3], [4], [5], [6] are viewed as one of the most promising atomic swap schemes, as they rely solely on cryptographic primitives to achieve cryptocurrency exchanges and do not require support for complex on-chain scripts from the underlying blockchains.

The universal atomic swap (UAS) construction [4] was first proposed for two-party exchanges, using *adaptor signatures* [7] and *verifiable timed dlog* (VTD) [8] instead of hashed timelock contracts (HTLC) [9]. ParaSwap [6] further extends UAS to support multi-party swaps across multiple blockchains. Roughly speaking, consider a simple three-party swap as illustrated in Fig. 1, where Alice intends to exchange amt_A for amt_B, Bob intends to exchange amt_B for amt_C, and Carol intends to exchange amt_C for amt_A. Each party is first required to compute a VTD, so that the locked coins can be refunded to the original owner after the corresponding timeout. Alice, Bob, and Carol then sequentially lock their coins into a shared address on their respective blockchains. Next, Alice samples a statement-witness pair (Y, y), and each party receives a pre-signature based on the same statement Y. UAS ensures if Alice redeems Carol's amt_C by revealing the witness y, then Carol and Bob can use the same y to redeem Bob's amt_B and Alice's amt_A in sequence. Otherwise, after the respective timeouts, each party reclaims its original coins by opening the corresponding VTD.

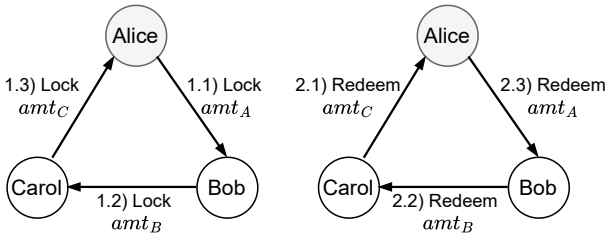


Fig. 1: An example of three-party swaps.

Griefing attacks [10], [11] arise when a deviating party unilaterally aborts the protocol, leading to the temporary lockup of other parties' assets until timeout. For example, Alice may refuse to redeem Carol's asset if its market price declines, or Carol may fail to lock his asset if Bob's asset depreciates. Although atomicity guarantees that either all parties receive the expected assets or all retain their original holdings, the locked assets may incur indirect economic costs during a waiting period, such as foregone interest or additional transaction fees.

To mitigate such attacks, several works [12], [13] proposed griefing-free swap protocols using premium-based mechanisms, where a deviating party's premium is transferred to the compliant party as compensation upon abortion. Further studies introduced fast settlement schemes [14], [15] and dynamic premium adjustments [16] to optimize the efficiency and fairness of premium-based mechanisms. Unfortunately, these

solutions rely on Turing-complete capabilities of blockchains and are thus incompatible with scriptless blockchains. Nadahalli et al. [17] developed a griefing-free two-party swap using only HTLC scripts. This approach can be adapted to the universal two-party setting by replacing hashlock and timelock scripts with adaptor signatures and VTD, as used in UAS [4]. However, it can not be naively extended to the multi-party swap setting, where a participant may simultaneously interact with both compliant and deviating counterparties. Alternative approaches [18], [19] rely on a third-party intermediary, which is presumed to remain honest and not abort the swap due to its economic and reputation incentives, but this reintroduces centralization risks such as single points of failure.

In this paper, we bridge this gap by proposing the first griefing-free universal multi-party swap protocol that requires only signature verification from the underlying blockchains.

A. Key Challenges

We identify three critical challenges in achieving griefing-free universal multi-party swaps, including the *timeout race attack*, the *premium escape attack*, and the *topological limitation*, as illustrated below.

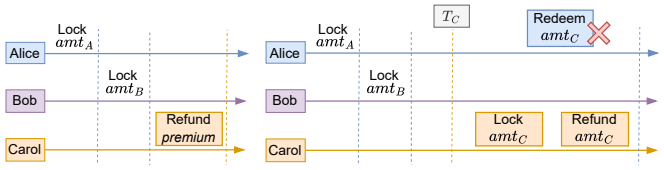
Timeout race attack. In HTLC-based protocols, timelock scripts impose strict expiration constraints to guarantee asset *safety*: if assets are not redeemed before the pre-determined timeout, the locked assets can *only* be refunded to their original owners, ensuring no compliant party loses assets. By contrast, universal protocols replace timelock scripts with verifiable timed dlog (VTD), which enforce only a lower bound on the opening time of refund signatures.

The absence of upper bound constraints opens the door to a novel vulnerability, which we term a *timeout race attack*: a deviating party can trigger a race condition by posting a pre-timeout transaction after the designated timeout, conflicting with a post-timeout transaction; if the pre-timeout transaction is confirmed, atomicity is compromised. For instance, consider the three-party swap in Fig. 1 with Alice's timeout T_A, Bob's timeout T_B, and Carol's timeout T_C, where $T_A > T_B > T_C$. A deviating Alice may delay redeeming Carol's amt_C until after Carol's timeout T_C, and Bob subsequently reclaims his amt_B after T_B; therefore, Carol can neither refund her amt_C nor redeem Bob's amt_B, ultimately violating atomicity.

PipeSwap [5] first identifies this attack in universal two-party swaps and introduces a two-hop completion mechanism to mitigate the timeout race attack[1]. It leverages the multi-input multi-output (MIMO) capability of UTXO-based blockchains to split Bob's coins into two outputs, ϵ and $amt_B - \epsilon$, where ϵ is designated to determine the final direction of the asset flow. However, this solution applies only to basic two-party scenarios and cannot be generalized to multi-party swaps.

In fact, we observe a straightforward yet effective countermeasure for mitigating the timeout race attack in multi-party swaps: extending each timeout interval to at least $2\delta + \varphi$, where δ and φ are the upper bound of network delivery

[1]Referred to as the *double-claiming attack* in PipeSwap [5].



(a) Premium escape attack (b) Time race attack

Fig. 2: Examples of attacks.

delay and confirmation latency, respectively, in the underlying blockchain. This setting ensures that any compliant party always has sufficient time (i.e., $\geq \delta + \varphi$) to redeem its expected coins even when a deviating party attempts to delay redemption. We do not claim any novelty for this interval setting, as it follows directly from the assumed bounds on network delay and confirmation latency.

Nevertheless, we note that although this solution mitigates the timeout race attack in single-timeout universal protocols, it does not extend to griefing-free swap protocols, which involve multiple overlapping refund timeouts on the same locked assets.

Premium escape attack. Xue and Herlihy [13] proposed a griefing-free solution for multi-party swaps based on a premium-based mechanism, which requires each party to lock two types of premiums: a *lock premium* and a *redemption premium*. The lock premium is awarded to the compliant party if the expected coins are not locked by a specified timeout. The redemption premium is awarded to the compliant party if its locked coins are not redeemed by a specified timeout. A smart contract enforces the premium distribution by verifying pre-specified on-chain actions and timeouts. Upon any deviation (e.g., refusal to redeem), the compliant party can receive the deviating party's premium from the contract as compensation.

However, universal protocols lack the expressive scripts to enforce correct premium transfers. As a result, a deviating party may refund its premium early and then abort the swap without incurring any penalty, even if a compliant party has locked its principal. We refer to this vulnerability as the *premium escape attack*. For instance, Fig. 2a shows that a deviating Carol fails to lock her amt_C but still reclaims her premium, thereby aborting the swap without losing her premium. We note that the core issue remains that the universal setting lacks an upper bound on the validity period of the premium refund transaction, enabling a deviating party to withdraw its premium early.

To resolve this issue, a straightforward approach is to use a revocable refund authorization that automatically expires after a predefined timeout. Unfortunately, existing revocation schemes [20], [21], [22], [23] typically rely on additional infrastructure and trust (e.g., a trusted revocation authority), which is fundamentally at odds with the security model of widely used public blockchains such as Ethereum and Bitcoin. To overcome the challenge in standard blockchain models, we propose an *asset migration mechanism*. The key idea is to transfer the newly locked asset together with the previously locked assets into a new address, such that once a new refund

transaction becomes valid for the previously locked assets, it invalidates any previously valid refund transaction.

However, while the asset migration mechanism can mitigate the premium escape attack, it may still be vulnerable to the timeout race attack. For example, Fig. 2b shows that a deviating Carol may delay locking amt_C until her timeout T_C, which conflicts with Alice's redemption, causing Alice to fail to redeem amt_C and lose her premium. To address such attacks, we extend the timeout interval of each lock transaction to be at least $2\delta + 2\varphi$. This setting ensures that even if a deviating party delays a lock transaction that conflicts with the compliant party's transaction, the compliant party either (i) obtains the deviating party's premium as compensation if it wins the transaction race, or (ii) retains sufficient time (i.e., $\geq \delta + \varphi$) to continue executing the protocol.

Topological limitation of universal multi-party swaps. Herlihy [24] explores multi-party swap models on arbitrary strongly connected digraphs. This method requires smart contracts capable of storing and operating on the entire swap digraph, as well as solving $\mathbb{NP}$-hard graph problems (e.g., finding long paths or a minimal feedback vertex set). In universal swap protocols, however, no on-chain script can perform such complex operations. ParaSwap [6] presents a universal multi-party swap protocol that parallelizes the lock and withdrawal steps, but it is restricted to single-leader digraphs and each party has only one incoming arc and one outgoing arc.

In our work, building on the impossibility result from [25], which shows that HTLC-based protocols cannot support atomic swaps in arbitrary multi-party swap digraphs and defines a specialized class of digraphs called *reuniclus graphs* for HTLC-based swaps, we prove that universal swap protocols based on adaptor signature (AS) and verifiable timed dlog (VTD) can achieve atomic swaps only on reuniclus graphs (see Appendix C). However, we note that unlike the strongly connected swap digraphs considered in Herlihy [24], reuniclus graphs feature two distinct types of leader vertices, i.e., main leaders and non-main leaders (as detailed in Section II-B), which are incompatible with the existing multi-party premium distribution mechanism [13]. In this work, we address this gap by designing a novel premium distribution mechanism that accommodates two types of leader vertices in reuniclus graphs.

Specifically, for each vertex v in a reuniclus graph, the lock premium guarantees that if any counterparty on an incoming arc to v fails to lock its principal, v can claim that counterparty's lock premium as compensation, which suffices to cover all lock premiums paid by v, since v cannot lock its principal on its outgoing arcs if it does not receive all incoming principal amounts. Likewise, the redemption premium guarantees that if any counterparty on an outgoing arc from v fails to redeem its principal, v can claim that counterparty's redemption premium, which suffices to cover all redemption premiums paid by v, since v cannot redeem its incoming principals if redemption fails on all of its outgoing arcs.

B. Our Contributions

The main contributions of this paper are summarized as follows.

- We propose GumSwap, the first griefing-free universal multi-party swap protocol that requires only basic signature verification from the underlying blockchains. GumSwap ensures a compliant party can receive compensation if its locked asset is not redeemed. We model this protocol within the universal composability framework [26], [27] and provide a formal security proof.
- We identify three challenges in achieving GumSwap: (i) the *timeout race attack*, which arises due to the absence of an upper bound on the transaction validity period; (ii) the *premium escape attack*, which arises from multiple refund transactions for the same assets being simultaneously valid in griefing-free protocols; and (iii) the *topological limitation* of universal multi-party swaps, which restricts swaps to a specialized class of digraphs called reuniclus graphs. To address these challenges, (i) we set a minimum timeout interval of $2\delta + \varphi$ for the principal timeout and a timeout interval of $2\delta + 2\varphi$ for the premium timeout to ensure any compliant party always has sufficient time to continue the protocol; (ii) we propose the asset migration mechanism to achieve at most one refund transaction for a given asset can be valid within a timeout interval; and (iii) we design a novel premium distribution mechanism tailored to reuniclus graphs, which feature two distinct types of leader vertices.
- We implement the GumSwap protocol and conduct extensive experiments to evaluate its off-chain and on-chain costs. The results demonstrate that our protocol is efficient and practical. Specifically, GumSwap achieves a running time of less than 0.5 seconds per party and communication costs of less than 865KB per party, while reducing gas costs by $10.3\times$ compared with existing contract-based solutions.

II. Preliminaries

In this section, we introduce the cryptographic primitives and digraph terminology used in this work.

A. Cryptographic Primitives

We now present the cryptographic primitives. Formal descriptions can be found in Appendix A.

Hard Relations. A hard relation $\mathcal{R}$ is defined by a statement-witness pair $(Y, y) \in \mathcal{R}$. We denote $L_{\mathcal{R}}$ as the language associated with $\mathcal{R}$, defined by $L_{\mathcal{R}} := \{Y \mid \exists y \text{ s.t. } (Y, y) \in \mathcal{R}\}$. A relation is considered hard if it satisfies: (i) There exists a PPT sampling algorithm $(Y, y) \leftarrow \mathsf{RGen}(1^\lambda)$ that outputs a statement-witness pair $(Y, y) \in \mathcal{R}$; (ii) The relation is polynomial-time decidable; (iii) For all PPT adversaries $\mathcal{A}$, the probability of $\mathcal{A}$ outputting a witness y on input Y is negligible.

Digital Signature. A signature scheme $\Sigma_{\mathsf{SIG}} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ consists of three algorithms: $(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$ generates a public-private key (pk, sk); the signing algorithm $\sigma \leftarrow \mathsf{Sign}(sk, m)$ takes sk

and a message m to produce a signature σ; $\mathsf{Vrfy}(pk, m, \sigma)$ checks if σ is valid, returning 1 if valid or 0 otherwise. Σ_{SIG} ensures *correctness* and *unforgeability*.

Adaptor Signature. An adaptor signature scheme [7] integrates a signature scheme Σ_{SIG} and a hard relation $\mathcal{R}$, incorporating four algorithms $\Sigma_{\mathsf{AS}} = (\mathsf{pSign}, \mathsf{Adapt}, \mathsf{pVrfy}, \mathsf{Ext})$: $\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$ generates a pre-signature $\tilde{\sigma}$ from a private key sk, a message $m \in \{0,1\}^*$, and a statement $Y \in L_{\mathcal{R}}$. $\mathsf{pVrfy}(pk, m, Y, \tilde{\sigma})$ verifies $\tilde{\sigma}$, returning 1 if valid or 0 otherwise. $\sigma \leftarrow \mathsf{Adapt}(\tilde{\sigma}, y)$ transforms $\tilde{\sigma}$ into a full signature σ using a witness y. $y \leftarrow \mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$ extracts y from σ and $\tilde{\sigma}$ such that $(Y, y) \in \mathcal{R}$. This scheme ensures the *unforgeability*, *pre-signature correctness*, *pre-signature adaptability*, and *witness extractability*.

Verifiable Timed Dlog. A verifiable timed dlog (discrete logarithm) [8] $\Sigma_{\mathsf{VTD}} = (\mathsf{Commit}, \mathsf{Verify}, \mathsf{Open}, \mathsf{ForceOp})$ involves four phases: $(C, \pi) \leftarrow \mathsf{Commit}(sk, T)$ inputs a secret value sk (generated using $\Sigma_{\mathsf{SIG}}.\mathsf{KGen}(1^\lambda)$) and a time parameter T, then produces a timed commitment C and a proof π. $\mathsf{Verify}(pk, C, \pi)$ checks the integrity of C and π, and accepts them by outputting 1 if sk embedded in C satisfies $pk = g^{sk}$; otherwise, it outputs 0. $(sk, r) \leftarrow \mathsf{Open}(C)$ allows the committer to reveal sk and the associated randomness r. $sk \leftarrow \mathsf{ForceOp}(C)$ extracts sk from C within the time T. A VTD scheme satisfies *soundness* and *privacy*.

Two-Party Computation. A two-party computation (2PC) protocol enables two parties to jointly compute a function over their private inputs while keeping these inputs hidden. In our setting, we use 2PC to realize three interactive protocols. The joint address creation $\Pi_{\mathsf{SIG}}^{\mathsf{JKGen}}$: on public parameters $(\mathbb{G}, g, q)$, parties v and u jointly generate a shared public key pk_{vu} and private shares sk_{vu}^v held by v and sk_{vu}^u held by u such that $pk_{vu} = g^{sk_{vu}}$, where $sk_{vu} = sk_{vu}^v \oplus sk_{vu}^u$ and $\oplus$ is $+$ for Schnorr and $\times$ for ECDSA. The joint signing $\Pi_{\mathsf{SIG}}^{\mathsf{JSign}}$: on public input a transaction tx and private inputs sk_{vu}^v, sk_{vu}^u, it outputs a joint signature σ_{tx} on tx. The joint pre-signing $\Pi_{\mathsf{AS}}^{\mathsf{JpSign}}$: on public input a transaction tx and a statement Y of a hard relation $\mathcal{R}$, and private inputs sk_{vu}^v, sk_{vu}^u, it outputs a pre-signature $\tilde{\sigma}_{tx}$ on tx. In this paper, we instantiate the 2PC protocols as those used in [5], [6].

B. Digraphs

Let $\mathcal{D} = (V, A)$ be a *directed graph* (*digraph*) with vertex set V and arc set A. Each arc (v, u) is directed from v to u. A *path* in $\mathcal{D}$ is a sequence of vertices such that each vertex (except the last) has an outgoing arc to the next without the arc repeated. A *cycle* is a path whose first and last vertices are the same. A *simple cycle* is a cycle where all vertices are distinct except the first and last. A digraph $\mathcal{C} = (V', A')$ is a *subgraph* of $\mathcal{D}$ if $V' \subseteq V$ and $A' \subseteq A$. $\mathcal{C}$ is called an *induced subgraph* of $\mathcal{D}$ if A' includes all arcs in A connecting vertices in V'. A digraph $\mathcal{D}$ is *strongly connected* if, for every pair of vertices v and u, there is a directed path from v to u and from u to v. A *feedback vertex set* is a subset $U \subseteq V$ such that removing all vertices in U from $\mathcal{D}$ results in an acyclic graph.

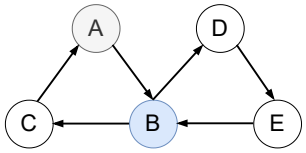


Fig. 3: A reuniclus graph with main leader A and non-main leader B.

Reuniclus Graphs. A strongly connected digraph $\mathcal{D}$ is called a *reuniclus graph* if there are vertices $(v_1, \ldots, v_p) \in \mathcal{D}$, induced subgraphs $(\mathcal{C}_1, \ldots, \mathcal{C}_p)$ of $\mathcal{D}$, and a rooted tree $\mathcal{T}$ whose nodes are these vertices $(v_1, \ldots, v_p)$, such that:

1) Each $\mathcal{C}_i$ is a single-leader subgraph with a leader ℓ_i. We call $\mathcal{C}_i$ a *leader component* of $\mathcal{D}$.
2) For any $i \neq j$,

$$\mathcal{C}_i \cap \mathcal{C}_j = \begin{cases} \{\ell_i\} & \text{if } \ell_j \text{ is the parent of } \ell_i \text{ in } \mathcal{T}, \\ \emptyset & \text{otherwise.} \end{cases}$$

The equation presents that two leader components $\mathcal{C}_i$ and $\mathcal{C}_j$ are either disjoint or share only one vertex, the leader ℓ_i of $\mathcal{C}_i$. $\mathcal{T}$ is referred to as the *control tree* of $\mathcal{D}$. If ℓ_j is the parent of ℓ_i in $\mathcal{T}$, then $\mathcal{C}_j$ is the *parent component* of ℓ_i , and $\mathcal{C}_i$ is its *child component* of ℓ_j. We assume the root component of $\mathcal{T}$ is $\mathcal{C}_1$, whose leader is called the *main leader* ℓ_1, while the leaders of non-root components $\mathcal{C}_i$ are called *non-main leaders* ℓ_i. We refer readers to [25] for a comprehensive definition.

For multi-party swaps on reuniclus graphs, the locking phase begins with each leader initiating contracts within its home component. A non-main leader proceeds to create contracts in its parent component only after all contracts in its home component are completed. The redemption phase then propagates in the reverse direction, starting from the main leader.

Fig. 3 illustrates a reuniclus graph, where A is the main leader, B is a non-main leader, and the remaining vertices are followers. The induced subgraphs $\mathcal{C}_1$ and $\mathcal{C}_2$ on $\{A, B, C\}$ and $\{B, D, E\}$ are single-leader components, respectively, where vertex A is the parent of B. Hence, $\mathcal{C}_1$ is the parent component of B and $\mathcal{C}_2$ is the home component of B. In swap protocols, the locking sequence follows $(A, B) \| (B, D) \rightarrow (D, E) \rightarrow (E, B) \rightarrow (B, C) \rightarrow (C, A)$, and the corresponding redemption sequence is $(C, A) \rightarrow (B, C) \rightarrow (A, B) \| (E, B) \rightarrow (D, E) \rightarrow (B, D)$, where the symbol $\|$ denotes that the operations are executed in parallel.

III. Formal Definition of GumSwap

In this section, we formalize the security model and the ideal functionality of GumSwap.

A. Security Model

We utilize the universal composability (UC) framework [26], specifically, the Global UC version [27], to formally model the security of GumSwap.

UC Security The protocol Π UC-realizes the functionality $\mathcal{F}$ if for every adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ such that for all environments $\mathcal{Z}$, the executions $\mathsf{EXEC}_{\Pi,\mathcal{A},\mathcal{Z}}$ and $\mathsf{EXEC}_{\mathcal{F},\mathcal{S},\mathcal{Z}}$ are indistinguishable to the environment $\mathcal{Z}$.

Adversary Model. We consider a static corruption model in which the adversary $\mathcal{A}$ can corrupt an arbitrary subset of participants at the beginning of the process. The corrupted parties may arbitrarily deviate from the protocol (e.g., refuse to transfer assets to its counterparty).

Communication Network. We adopt the same network assumptions as in prior works [4], [5]. We assume a synchronous communication network, and the execution of the protocol occurs in discrete rounds. This is modeled by the global clock functionality $\mathcal{F}_{clock}$ [28], where all honest parties are required to indicate they are ready to move to the next round before the clock can proceed. A party can abort at any round by sending a special abort message. We also assume the existence of secure message transmission channels, modeled by the ideal functionality $\mathcal{F}_{smt}$, ensuring the adversary cannot read or alter the contents of messages.

Blockchain Functionalities Consistent with prior works [4], [5], we formalize the underlying blockchain as a global functionality $\mathcal{F}_B$ with δ-synchronous communication and φ-confirmation time. An adversary $\mathcal{A}_B$ may delay or reorder transactions within δ rounds, but cannot modify or drop them, and any delivered transaction can be recorded within φ time units. $\mathcal{F}_B$ maintains a ledger $\mathcal{L}$ of balances $pk.\mathsf{bal}$ and an append-only bulletin board $\mathcal{T}$ that stores submitted transactions tx with timestamps t. $\mathcal{F}_B$ provides three interfaces: (i) $\mathsf{Valid}(tx)$ checks transaction validity (e.g., sufficient balance in $\mathcal{L}$, correct signatures, and no conflict with $\mathcal{T}$). (ii) $\mathsf{Publish}(tx, t)$ appends a valid tx to $\mathcal{T}$ at time $t' \leq t + \delta$, where the specific time t' is determined by the ideal adversary $\mathcal{S}_B$; if conflicting tx_0, tx_1 with equal transaction fees are published at the same t', $\mathcal{T}$ selects one uniformly at random. (iii) $\mathsf{Confirm}(tx)$ updates $\mathcal{L}$ by transferring coins from the input account pk_{in} to the output account pk_{out} after tx has remained in $\mathcal{T}$ for φ time. We refer the reader to [5] for a formal definition of this functionality.

B. Formal Functionality

We formalize the security of GumSwap using an ideal functionality $\mathcal{F}_{\mathsf{GS}}$, illustrated in Fig. 4. The functionality $\mathcal{F}_{\mathsf{GS}}$ interacts with multiple users, a simulator $\mathcal{S}$, and multiple global blockchain functionalities. It maintains a list Locked that stores the locked coins in the functionality, initialized to $\emptyset$. For each $(v, u) \in \mathcal{D}$, it also maintains state variables $b^L_{vu}, b^R_{vu}, b^P_{vu}$, which represent the states of the lock premium p^L_{vu}, the redemption premium p^R_{vu}, the principal amt_v, respectively; all are initialized to $\perp$. $\mathcal{F}_{\mathsf{GS}}$ specifies four procedures as follows, each triggered by a message that includes a request and a session identifier sid.

Lock premium setup. Each leader $v = \ell_i$ first initiates a lock premium deposit with a message (lock-pre, sid, PK_v, PK_u, SK_v, P^L_{vu}) to deposit each $p^L_{vu} \in P^L_{vu}$ on the corresponding outgoing arc $(v, u) \in \mathcal{C}_i$. Each non-main leader $v = \ell_i$ with $i \neq 1$ sends this message on each outgoing arc $(v, u) \in \mathcal{C}_j$ only if $b^L_{wv} = 1$ for both $(w, v) \in \mathcal{C}_i$ and $(w, v) \in \mathcal{C}_j$. Each follower $v \neq \ell_i$ also sends this message on each outgoing arc $(v, u) \in \mathcal{D}$ only if $b^L_{wv} = 1$ for all $(w, v) \in \mathcal{D}$. For each (v, u), $\mathcal{F}_{\mathsf{GS}}$ transfers p^L_{vu} from account $pk_v \in PK_v$ to a specific account $pk^{L,v}_{\mathcal{F}}$ controlled by $\mathcal{F}_{\mathsf{GS}}$, sets $b^L_{vu} = 1$, and appends the corresponding entry to the list Locked. After T^L_v, if the assets are still locked, then $\mathcal{F}_{\mathsf{GS}}$ refunds p^L_{vu} to party v, sets $b^L_{vu} = 0$, and removes this entry from Locked.

Redemption premium setup. The main leader $v = \ell_1$ sends a message (redp-pre, sid, PK_v, PK_w, SK_v, P^R_{wv}) to deposit each $p^R_{wv} \in P^R_{wv}$ on the corresponding incoming arc $(w, v) \in \mathcal{D}$ only if $b^L_{wv} = 1$ for all $(w, v) \in \mathcal{D}$. Each non-main leader $v = \ell_i$ with $i \neq 1$ sends this message on each incoming arc $(w, v) \in \mathcal{C}_i$ and $(w, v) \in \mathcal{C}_j$ only if $b^R_{vu} = 1$ for some $(v, u) \in \mathcal{C}_j$. Each follower $v \neq \ell_i$ sends this message on each incoming arc $(w, v) \in \mathcal{D}$ only if $b^R_{vu} = 1$ for some $(v, u) \in \mathcal{D}$. For each (w, v), $\mathcal{F}_{\mathsf{GS}}$ transfers p^R_{wv} together with previously locked p^L_{wv} to an account $pk^{R,v}_{\mathcal{F}}$, sets $b^R_{wv} = 1$, and appends the corresponding entry to the list Locked. After T^R_v, if the coins $p^L_{wv} + p^R_{wv}$ are still locked, $\mathcal{F}_{\mathsf{GS}}$ refunds them to party v, sets $b^R_{wv} = 0$, and removes this entry from Locked.

Swap lock. Each leader $v = \ell_i$ sends a message (lock, sid, PK_v, SK_v, AMT_v) to lock each $amt_v \in AMT_v$ on the corresponding outgoing arc $(v, u) \in \mathcal{C}_i$ only if $b^R_{vu} = 1$ for any $(v, u) \in \mathcal{C}_i$. Each non-main leader $v = \ell_i$ with $i \neq 1$ sends this message on each $(v, u) \in \mathcal{C}_j$ only if $b^P_{wv} = 1$ for all $(w, v) \in \mathcal{C}_i$ and $(w, v) \in \mathcal{C}_j$. Each follower $v \neq \ell_i$ sends this message on each $(v, u) \in \mathcal{D}$ only if $b^P_{wv} = 1$ for all $(w, v) \in \mathcal{D}$. For each (v, u), $\mathcal{F}_{\mathsf{GS}}$ transfers amt_v together with previously locked p^R_{vu} to an account $pk^{P,v}_{\mathcal{F}}$, and refunds p^L_{vu} to party v. It sets $b^P_{vu} = 1$ and appends the corresponding entry to the list Locked. After T^P_v, if the coins $amt_v + p^R_{vu}$ are still locked, then $\mathcal{F}_{\mathsf{GS}}$ refunds them to party v, sets $b^P_{vu} = 0$, and removes this entry from Locked.

Swap complete. The main leader $v = \ell_1$ sends a message $(\mathsf{swap}, sid, PK_v)$ to redeem the assets on its incoming arcs $(w, v) \in \mathcal{D}$ only if $b^P_{wv} = 1$ for all $(w, v) \in \mathcal{D}$. Each non-main leader $v = \ell_i$ with $i \neq 1$ sends this message on each arc $(w, v) \in \mathcal{C}_i$ and $(w, v) \in \mathcal{C}_j$ only if $b^P_{vu} = 0$ for some $(v, u) \in \mathcal{C}_j$. Each follower $v \neq \ell_i$ sends this message on each arc $(w, v) \in \mathcal{D}$ only if $b^P_{vu} = 0$ for some $(v, u) \in \mathcal{D}$. For each (w, v), $\mathcal{F}_{\mathsf{GS}}$ transfers $amt_w + p^R_v$ to pk_v, and sets $b^P_{wv} = 0$.

IV. GumSwap Protocol

In this section, we first present the premium distribution mechanism on reuniclus graphs and then illustrates GumSwap protocol. Finally, we provide a formal security analysis of GumSwap. Before illustrating the protocols, we first specify the system model, threat model, network assumptions, and security properties.

System Model. We model a multi-party swap as a strongly connected directed graph $\mathcal{D}$, where each vertex represents a party and each directed arc represents a proposed asset transfer. We write $(v, u) \in \mathcal{D}$ to denote that (v, u) is an arc of $\mathcal{D}$. Party v owns the assets associated with its outgoing arcs (v, u), and intends to securely obtain the assets associated with its incoming arcs (w, v). We assume participants possess a bootstrapping mechanism, such as a forum or bulletin board,

Lock Premium Setup Phase

Upon receiving $(\mathsf{lock\text{-}pre}, sid, PK_v, PK_u, SK_v, P^L_{vu})$ from $v \in \mathcal{D}$ for some $(v,u) \in \mathcal{D}$, where $PK_v = \left(pk_v\right)_{v\in(v,u)}$, $PK_u := \left(pk_u\right)_{u\in(v,u)}$, $SK_v := \left(sk_v\right)_{v\in(v,u)}$, and $P^L_{vu} := \left(p^L_{vu}\right)_{v\in(v,u)}$,

1. Check if (i) $v = \ell_i$ and $(v,u) \in \mathcal{C}_i$, (ii) $v = \ell_i$ with $i \neq 1$, $(v,u) \in \mathcal{C}_j$, and $b^L_{wv} = 1$ for all $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$; or (iii) $v \neq \ell_i$, $(v,u) \in \mathcal{D}$, and $b^L_{wv} = 1$ for all $(w,v) \in \mathcal{D}$, then continue; otherwise, abort.
2. For each (v,u), invoke subroutine $\mathsf{Freeze}(sid, pk_v, pk^{L,v}_{\mathcal{F}}, p^L_{vu}, T^L_v)$.
3. For each (v,u), upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_B$, append the entry $(sid, pk_v, pk^{L,v}_{\mathcal{F}}, p^L_{vu}, T^L_v)$ to the list Locked, set $b^L_{vu} = 1$, and send (locked, ok) to all parties.
4. After time T^L_v, if the entry $(sid, pk_v, pk^{L,v}_{\mathcal{F}}, p^L_{vu}, T^L_v)$ is still in Locked, then invoke subroutine $\mathsf{Unfreeze}(sid, pk^{L,v}_{\mathcal{F}}, pk_v, p^L_{vu})$, set $b^L_{vu} = 0$, and delete the entry from Locked.

Redemption Premium Setup Phase

Upon receiving $(\mathsf{redp\text{-}pre}, sid, PK_v, PK_w, SK_v, P^R_{wv})$ from $v \in \mathcal{D}$ for some $(w,v) \in \mathcal{D}$, where $PK_v = \left(pk_v\right)_{v\in(w,v)}$, $PK_w := \left(pk_w\right)_{w\in(w,v)}$, $SK_v := \left(sk_v\right)_{v\in(w,v)}$, and $P^R_{wv} := \left(p^R_{wv}\right)_{v\in(w,v)}$,

1. Check if (i) $v = \ell_1$ and $b^L_{wv} = 1$ for all $(w,v) \in \mathcal{D}$, (ii) $v = \ell_i$ with $i \neq 1$, $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$, and $b^R_{vu} = 1$ for some $(v,u) \in \mathcal{C}_j$, or (iii) $v \neq \ell_i$, $(w,v) \in \mathcal{D}$, and $b^R_{vu} = 1$ for some $(v,u) \in \mathcal{D}$, then continue; otherwise, abort.
2. For each (w,v), invoke subroutines $\mathsf{Freeze}(sid, pk_v, pk^{R,v}_{\mathcal{F}}, p^R_{wv}, T^R_v)$ and $\mathsf{Freeze}(sid,\ pk^{L,w}_{\mathcal{F}},\ pk^{R,v}_{\mathcal{F}},\ p^L_{wv},\ T^R_v)$.
3. For each (w,v), upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_B$, append the entry $(sid, pk_v, pk^{R,v}_{\mathcal{F}}, p^R_{wv}+p^L_{wv}, T^R_v)$ to the list Locked, delete the entry $(sid, pk_w,\ pk^{L,w}_{\mathcal{F}},\ p^L_{wv},\ T^L_w)$, set $b^R_{wv} = 1$, and send (locked, ok) to all parties.
4. After time T^R_v, if the entry $(sid, pk_v, pk^{R,v}_{\mathcal{F}}, p^R_{wv}+p^L_{wv}, T^R_v)$ is still in Locked, then invoke subroutine $\mathsf{Unfreeze}(sid,\ pk^{R,v}_{\mathcal{F}},\ pk_v,\ p^R_{wv}+p^L_{wv})$, set $b^R_{wv} = 0$, and delete the entry from Locked.

Swap Lock Phase

Upon receiving $(\mathsf{lock}, sid, PK_v, SK_v, AMT_v)$ from $v \in \mathcal{D}$ for some $(v,u) \in \mathcal{D}$, where $PK_v = \left(pk_v\right)_{v\in(v,u)}$, $SK_v := \left(sk_v\right)_{v\in(v,u)}$, and $AMT_v := \left(amt_v\right)_{v\in(v,u)}$,

1. Check if (i) $v = \ell_i$, $(v,u) \in \mathcal{C}_i$, and $b^R_{vu} = 1$ for some $(v,u) \in \mathcal{D}$; (ii) $v = \ell_i$ with $i \neq 1$, $(v,u) \in \mathcal{C}_j$, and $b^P_{wv} = 1$ for all $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$; or (iii) $v \neq \ell_i$ and $b^P_{wv} = 1$ for all $(w,v) \in \mathcal{D}$. If none of the above conditions holds, abort.
2. For each (v,u), invoke subroutines $\mathsf{Freeze}(sid,\ pk_v,\ pk^{P,v}_{\mathcal{F}},\ amt_v,\ T^P_v)$, $\mathsf{Freeze}(sid,\ pk^{R,u}_{\mathcal{F}},\ pk^{P,v}_{\mathcal{F}},\ p^R_{vu},\ T^P_v)$, and $\mathsf{Unfreeze}(sid,\ pk^{R,u}_{\mathcal{F}},\ pk_v,\ p^L_{vu})$.
3. For each (v,u), upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_{B_v}$, append the entry $(sid, pk_v, pk^{P,v}_{\mathcal{F}}, amt_v+p^R_{vu}, T^P_v)$ to the list Locked, delete the entry $(sid, pk_v, pk^{R,v}_{\mathcal{F}}, p^R_{vu}+p^L_{vu}, T^R_v)$, set $b^P_{vu} = 1$, and send (locked, ok) to all parties.
4. After time T^P_v, if the entry $(sid, pk_v, pk^{P,v}_{\mathcal{F}}, amt_v+p^R_{vu}, T^P_v)$ is still in Locked, invoke subroutine $\mathsf{Unfreeze}(sid,\ pk^{P,v}_{\mathcal{F}},\ pk_v,\ amt_v+p^R_{vu})$, set $b^P_{vu} = 0$, and delete the entry from Locked.

Swap Complete Phase

Upon receiving $(\mathsf{swap}, sid, PK_v)$ from v for some $(w,v) \in \mathcal{D}$, where $PK_v = \left(pk_v\right)_{v\in(w,v)}$,

1. Check if (i) $v = \ell_1$ and $b^P_{wv} = 0$ for all $(w,v) \in \mathcal{D}$, (ii) $v = \ell_i$ with $i \neq 1$, $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$, and $b^P_{vu} = 0$ for some $(v,u) \in \mathcal{C}_j$, or (iii) $v \neq \ell_i$, $(w,v) \in \mathcal{D}$, and $b^P_{vu} = 0$ for any $(v,u) \in \mathcal{D}$, then continue; otherwise, abort.
2. For each (w,v), invoke subroutine $\mathsf{Transfer}(sid, pk^{P,w}_{\mathcal{F}}, pk_v, amt_w+p^R_{wv})$.
3. For each (w,v), upon receiving $(\mathsf{confirmed}, sid)$ from $\mathcal{F}_B$, delete the entry $(sid, pk_v, pk^{P,w}_{\mathcal{F}}, amt_w+p^R_{wv}, T^P_w)$, set $b^P_{wv} = 0$, and send (swap, ok) to all parties.

Subroutine Descriptions:

Freeze$(sid, pk, pk_{\mathcal{F}}, v, T)$: Transfer coins v from account pk to $pk_{\mathcal{F}}$ (controlled by $\mathcal{F}$) via a transaction tx_{lock} and invoking interface $\mathsf{Confirm}(tx_{lock})$ of blockchain functionality $\mathcal{F}_B$. If tx_{lock} has been confirmed by $\mathcal{F}_B$, then respond $(\mathsf{confirmed}, sid)$.

Transfer$(sid, pk_{\mathcal{F}}, pk, v)$: transfer frozen coins v from account $pk_{\mathcal{F}}$ to pk by a transaction tx_{swp} and invoking interface $\mathsf{Confirm}(tx_{swp})$ of blockchain functionality $\mathcal{F}_B$. If tx_{swp} has been confirmed by $\mathcal{F}_B$, then respond $(\mathsf{confirmed}, sid)$.

Unfreeze$(sid, pk_{\mathcal{F}}, pk, v)$: transfer frozen coins v from account $pk_{\mathcal{F}}$ to pk by a transaction tx_{rfd} and invoking interface $\mathsf{Confirm}(tx_{rfd})$ of blockchain functionality $\mathcal{F}_B$. If tx_{rfd} has been confirmed by $\mathcal{F}_B$, then respond $(\mathsf{confirmed}, sid)$.

Fig. 4: Ideal Functionality $\mathcal{F}_{\mathsf{GS}}$

to discover counterparties and match demands, as assumed in prior works [6].

Threat model. Consistent with prior works [6], [24], we consider a Byzantine adversary that can corrupt an arbitrary subset of participants. Corrupted parties may arbitrarily deviate from the protocol to maximize their utility. For example, a corrupted party may receive assets while refusing to transfer its assets to others, or may collude with others to steal assets from compliant parties. We do not consider adversaries who compromise underlying blockchains themselves, such as by denial-of-service, long-range, and consensus-level attacks. We also assume miners do not censor or delay transactions.

Network assumptions. We adopt the same network assumptions as prior works [4], [5]: (i) Communication between parties is δ-synchronous, i.e., the network delivery delay is under adversarial control up to a known upper bound δ. (ii) Blockchains operate under a maximum confirmation delay φ, i.e., a valid transaction can be finally confirmed (i.e., cannot be reverted, replaced, or deleted) within time φ. If two conflicting transactions tx_1 and tx_2 with identical transaction fees become visible at times t_1 and t_2, respectively, with $t_1 < t_2$, miners always include tx_1. If they become visible simultaneously, i.e., $t_1 = t_2$, miners choose uniformly at random, so each of tx_1 and tx_2 is confirmed with probability $1/2$. (iii) All communication channels between participants are authenticated to prevent impersonation and ensure message integrity. impersonation and ensure message integrity.

Security Properties. GumSwap should satisfy the following properties:

- *Atomicity.* If a compliant party transfers its asset to the counterparty, it receives the counterparty's asset; otherwise, it recovers own asset.
- *Hedge.* If a compliant party locks assets that are not redeemed, it receives sufficient compensation for the locked assets.

A. Premium Distribution on Reuniclus Graphs

We now present a premium distribution mechanism on reuniclus graphs, incorporating *lock premium* and *redemption premium*. As illustrated in Section II-B, we denote that the root component of is $\mathcal{C}_1$, and its leader ℓ_1 is called the *main leader*. If a leader ℓ_i belongs to two components, $\mathcal{C}_i$ and $\mathcal{C}_j$, we say that $\mathcal{C}_i$ is *home component* of ℓ_i and $\mathcal{C}_j$ is *parent component* of ℓ_i. All non-leader vertices are called *followers*.

A redemption premium on an arc (v,u) is denoted by p^R_{vu}. It starts at the main leader ℓ_1 and moving against the orientation of the arcs. We assume each asset is assigned a uniform premium p.

$$p^R_{vu} = \begin{cases} p & \text{if } v = \ell_1 \\ p + \sum_{(w,v)\in\mathcal{C}_j} p^R_{wv} & \text{if } v = \ell_i, \ell_i \neq \ell_1 \ \&\ (v,u) \in \mathcal{C}_i \\ p + \sum_{(w,v)\in\mathcal{C}_i} p^R_{wv} & \text{if } v = \ell_i, \ell_i \neq \ell_1 \ \&\ (v,u) \in \mathcal{C}_j \\ p + \sum_{(w,v)\in\mathcal{D}} p^R_{wv} & \text{otherwise} \end{cases} \tag{1}$$

Eq. 1 shows the redemption premium distribution on reuniclus graphs, detailed as follows.

1. For the main leader $v = \ell_1$, each outgoing arc (v,u) is allocated an initial redemption premium p.
2. Each non-main leader $v = \ell_i$ $(i \neq 1)$ redeems the assets on $(w,v) \in \mathcal{C}_i$ only if the assets on $(v,u) \in \mathcal{C}_j$ have been redeemed. Thus, for each $v = \ell_i$ $(i \neq 1)$,
   - If $(v,u) \in \mathcal{C}_i$, p^R_{vu} comprises a premium p plus all redemption premiums on $(w,v) \in \mathcal{C}_j$.
   - If $(v,u) \in \mathcal{C}_j$, p^R_{vu} comprises a premium p plus all redemption premiums on $(w,v) \in \mathcal{C}_i$.
3. For each follower $v \neq \ell_i$, each (v,u) is allocated p^R_{vu} that comprises a premium p plus all redemption premiums on $(w,v) \in \mathcal{D}$.

Party v "activates" u's redemption premium p^R_{vu} upon locking its principal on (v,u); thereafter, v can claim p^R_{vu} if u fails to redeem v's principal.

A lock premium on an arc (v,u) is denoted by p^L_{vu}, which propagates forward in the digraph.

$$p^L_{wv} = \begin{cases} \sum_{(w,v)\in\mathcal{D}} p^R_{wv} & \text{if } v = \ell_1 \\ \sum_{(w,v)\in\mathcal{C}_j} p^L_{wv} + \sum_{(v,u)\in\mathcal{C}_i} p^R_{vu} & \text{if } v = \ell_i,\ \ell_i \neq \ell_1\ \&\ (w,v) \in \mathcal{C}_i \\ \sum_{(v,u)\in\mathcal{C}_j} p^L_{vu} + \sum_{(w,v)\in\mathcal{C}_i} p^R_{wv} & \text{if } v = \ell_i,\ \ell_i \neq \ell_1\ \&\ (w,v) \in \mathcal{C}_j \\ \sum_{(v,u)\in\mathcal{D}} p^L_{vu} & \text{otherwise} \end{cases} \tag{2}$$

Eq. 2 describes the lock premium distribution on reuniclus graphs, detailed as follows.

1. For the main leader $v = \ell_1$, each incoming arc (w,v) is allocated a lock premium p^L_{wv} equal to the sum of redemption premiums on $(w,v) \in \mathcal{D}$.
2. Each non-main leader $v = \ell_i$ $(i \neq 1)$ first locks all assets on $(v,u) \in \mathcal{C}_i$, and then lock the assets on $(v,u) \in \mathcal{C}_j$ only if the assets on incoming arcs in both $\mathcal{C}_i$ and $\mathcal{C}_j$ have been locked. Thus, for each $v = \ell_i$ $(i \neq 1)$,
   - If $(w,v) \in \mathcal{C}_i$, p^L_{wv} covers all lock premiums on $(w,v) \in \mathcal{C}_j$ and all redemption premiums on $(v,u) \in \mathcal{C}_j$;
   - If $(w,v) \in \mathcal{C}_j$, p^L_{wv} covers all lock premiums on $(v,u) \in \mathcal{C}_j$ and all redemption premiums on $(w,v) \in \mathcal{C}_i$.
3. For each follower $v \neq \ell_i$, each incoming arc (w,v) is allocated p^L_{wv} that covers all lock premiums on $(v,u) \in \mathcal{D}$.

Party v "activates" w's lock premium p^L_{wv} upon depositing the redemption premium p^R_{wv} on (w,v); thereafter, v can claim p^L_{wv}, if w fails to lock its principal.

B. Description of GumSwap Protocol

We now describe the GumSwap protocol based on Schnorr and ECDSA signature schemes.

GumSwap involves four phases: (1) lock premium setup, (2) redemption premium setup, (3) swap lock, and (4) swap complete phases. Similar to contract-based protocols, where all steps are hard-coded in contracts before any assets are locked, GumSwap requires that all shared addresses are created and pre-specific signatures are generated before parties deposit their premiums. For clarity, we use the terms *deposit premium*

Global Input: For each arc $(v,u) \in \mathcal{D}$: coin amounts $(p^L_{vu}, p^R_{vu}, amt_v) \in \mathbb{N}$; public keys: (pk_v, pk_u); time parameters: (T^L_v, T^R_u, T^P_v).

Private Input: For $(v,u) \in \mathcal{D}$, participant v's input: sk_v; participant u's input: sk_u.

Timeout Configuration: Let t denote the initial time where the main leader ℓ_1 deposits its lock premium.

For $v = \ell_1$, set $T^L_v = t + 3\,diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)$, $T^R_v = t + 5\,diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)$, and $T^P_v = t + 6\,diam(\mathcal{D}_\ell) \cdot (\delta + \varphi) + diam(\mathcal{D}_\ell) \cdot \delta$.

For $v = \ell_i$, $i \neq 1$, set $T^L_v = t + (diam(\mathcal{D}_\ell) + 2\mathcal{D}(v, \ell_1)) \cdot (\delta + \varphi)$ on $(v,u) \in \mathcal{C}_j$, $T^L_v = t + 3\,diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)$ on $(v,u) \in \mathcal{C}_i$, $T^R_v = t + (3\,diam(\mathcal{D}_\ell) + 2\mathcal{D}_{\ell_i}(v)) \cdot (\delta + \varphi)$ on $(v,u) \in \mathcal{C}_i$ and $(v,u) \in \mathcal{C}_j$, $T^P_v = t + (5\,diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)) + (\mathcal{D}(v, \ell_1) \cdot (2\delta + \varphi))$ on $(v,u) \in \mathcal{C}_j$, and $T^P_v = t + 6\,diam(\mathcal{D}_\ell) \cdot (\delta + \varphi) + diam(\mathcal{D}_\ell) \cdot \delta$ on $(v,u) \in \mathcal{C}_i$.

For $v \neq \ell_i$, set $T^L_v = t + (diam(\mathcal{D}_\ell) + 2\mathcal{D}(v, \ell_1)) \cdot (\delta + \varphi)$, $T^R_v = t + (3\,diam(\mathcal{D}_\ell) + 2\mathcal{D}_{\ell_i}(v)) \cdot (\delta + \varphi)$, and $T^P_v = t + (5\,diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)) + (\mathcal{D}(v, \ell_1) \cdot (2\delta + \varphi))$.

Lock Premium Setup Phase

Each party v prepares the pre-specified transactions and VTD commitments, and deposits the lock premium on each outgoing arc $(v,u) \in \mathcal{D}$. For all pairs of arcs (w,v) and (v,u), parties v, u, and w do the following:

1. Parties v and u execute the joint address generation protocol $\Pi^{\mathsf{JKGen}}_{\mathsf{SIG}}$ with the common input $(\mathbb{G}, g, q)$, and output $\left(pk^L_{vu}, pk^P_{vu}\right)$ to both parties, $\left(sk^{L,v}_{vu}, sk^{P,v}_{vu}\right)$ to party v, $\left(sk^{L,u}_{vu}, sk^{P,u}_{vu}\right)$ to party u.
2. Parties v and w execute the $\Pi^{\mathsf{JKGen}}_{\mathsf{SIG}}$ protocol with the input $(\mathbb{G}, g, q)$, and output pk^R_{wv} to both parties, $sk^{R,v}_{wv}$ to v, $sk^{R,w}_{wv}$ to w.
3. Party v generates (i) a redemption premium deposit transaction $tx^{R,v}_{lock} := \left(\left(pk_v, pk^L_{wv}\right), pk^R_{wv}, p^L_{wv}+p^R_{wv}\right)$, which transfers coins p^R_{wv} from address pk_v and coins p^L_{wv} from address pk^L_{wv} into address pk^R_{wv}, (ii) a principal lock transaction $tx^{P,v}_{lock} := \left(\left(pk_v, pk^R_{vu}\right), \left(pk^P_{vu}, pk_v\right), \left(amt_v+p^R_{vu}, p^L_{vu}\right)\right)$, which transfers amt_v from pk_v and p^R_{vu} from pk^R_{vu} into address pk^P_{vu}, and refunds p^L_{vu} to pk_v, and (iii) a swap transaction $tx^{wv}_{swp} := \left(pk^P_{wv}, pk_v, amt_w+p^R_{wv}\right)$, which transfers $amt_w+p^R_{wv}$ from address pk^P_{wv} to pk_v.
4. Party v computes $\left(C^{L,w}_{wv}, \pi^{L,w}_{wv}\right) \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{Commit}\left(sk^{L,v}_{wv}, T^L_w\right)$, $\left(C^{R,u}_{vu}, \pi^{R,u}_{vu}\right) \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{Commit}\left(sk^{R,v}_{vu}, T^R_u\right)$, and $\left(C^{P,w}_{wv}, \pi^{P,w}_{wv}\right) \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{Commit}\left(sk^{P,v}_{wv}, T^P_w\right)$.
5. Party v sends $tx^{P,v}_{lock}$ and $\left(C^{R,u}_{vu}, \pi^{R,u}_{vu}\right)$ to u, and sends $\left(tx^{R,v}_{lock}, tx^{wv}_{swp}\right)$, $\left(C^{L,w}_{wv}, \pi^{L,w}_{wv}\right)$, and $\left(C^{P,w}_{wv}, \pi^{P,w}_{wv}\right)$ to w.
6. Parties u and w check if their respective received transactions $\left(tx^{L,v}_{lock}, tx^{R,v}_{lock}, tx^{wv}_{swp}\right)$ are well formed (e.g., satisfying premium distribution mechanism) and $\Sigma_{\mathsf{VTD}}.\mathsf{Vrfy}\left(pk^{L,v}_{wv}, C^{L,w}_{wv}, \pi^{L,w}_{wv}\right) = 1$, $\Sigma_{\mathsf{VTD}}.\mathsf{Vrfy}\left(pk^{R,u}_{vu}, C^{R,u}_{vu}, \pi^{R,u}_{vu}\right) = 1$, $\Sigma_{\mathsf{VTD}}.\mathsf{Vrfy}\left(pk^{P,w}_{wv}, C^{P,w}_{wv}, \pi^{P,w}_{wv}\right) = 1$; otherwise, abort.
7. Each leader ℓ_i generates $(Y_i, y_i) \in \mathcal{R}$ and sends Y_i to participants in its home component $\mathcal{C}_i$.
8. Parties v and u run $\Pi^{\mathsf{JSign}}_{\mathsf{SIG}}$ with input $\left(sk^{R,v}_{vu}, sk^{R,u}_{vu}, tx^{P,v}_{lock}\right)$ to obtain signatures $\sigma^{R,vu}_{lock}$ under the address pk^R_{vu}.
9. Parties v and w ruwn $\Pi^{\mathsf{JSign}}_{\mathsf{SIG}}$ with input $\left(sk^{L,v}_{wv}, sk^{L,w}_{wv}, tx^{R,v}_{lock}\right)$ to obtain signatures $\sigma^{R,wv}_{lock}$ under the address pk^L_{wv}, and then run $\Pi^{\mathsf{JpSign}}_{\mathsf{AS}}$ with inputs $\left(sk^{P,v}_{wv}, sk^{P,w}_{wv}, tx^{wv}_{swp}, Y_i\right)$ and obtains a pre-signature $\tilde{\sigma}^{wv}_{swp}$ under the address pk^P_{wv}.
10. Party v generates a lock premium deposit transaction $tx^{L,v}_{lock} := \left(pk_v, pk^L_{vu}, p^L_{vu}\right)$ and computes $\sigma^{L,v}_{lock} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\left(sk_v, tx^{L,v}_{lock}\right)$. According to the role of party v in the digraph $\mathcal{D}$, v proceeds as follows.
    - If $v = \ell_i$, at time t, v posts $\left(tx^{L,v}_{lock}, \sigma^{L,v}_{lock}\right)$ on its outgoing arcs $(v,u) \in \mathcal{C}_i$.
    - If $v = \ell_i$, $i \neq 1$, at time $t + \mathcal{D}_{\ell_i}(v) \cdot (\delta + \varphi)$, v checks if all transactions $tx^{L,w}_{lock}$ have been confirmed on its incoming arcs $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$, and then v posts $\left(tx^{L,v}_{lock}, \sigma^{L,v}_{lock}\right)$ on its outgoing arcs $(v,u) \in \mathcal{C}_j$.
    - If $v \neq \ell_i$, at time $t + \mathcal{D}_{\ell_i}(v) \cdot (\delta + \varphi)$, v checks if all transactions $tx^{L,w}_{lock}$ have been confirmed on its incoming arcs $(w,v) \in \mathcal{D}$, v then posts $\left(tx^{L,v}_{lock}, \sigma^{L,v}_{lock}\right)$ on its outgoing arcs $(v,u) \in \mathcal{D}$.

Redemption Premium Setup Phase

For each $(w,v) \in \mathcal{D}$, v computes $\sigma^{R,v}_{lock} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\left(sk_v, tx^{R,v}_{lock}\right)$ under pk_v. According to the role of v in $\mathcal{D}$, v proceeds as follows.

- If $v = \ell_1$, at time $t + diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)$, v checks if all $tx^{L,w}_{lock}$ have been confirmed on its incoming arcs $(w,v) \in \mathcal{D}$, and then v posts $\left(tx^{R,v}_{lock}, \sigma^{R,v}_{lock}, \sigma^{R,wv}_{lock}\right)$ on its incoming arcs $(w,v) \in \mathcal{D}$.
- If $v = \ell_i$, $i \neq 1$, at time $t + (diam(\mathcal{D}_\ell) + 2\mathcal{D}(v, \ell_1)) \cdot (\delta + \varphi)$, v checks if at least one transaction $tx^{R,u}_{lock}$ has been confirmed on $(v,u) \in \mathcal{C}_j$, and then v posts $\left(tx^{R,v}_{lock}, \sigma^{R,v}_{lock}, \sigma^{R,wv}_{lock}\right)$ on both $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$.
- If $v \neq \ell_i$, at time $t + (diam(\mathcal{D}_\ell) + 2\mathcal{D}(v, \ell_1)) \cdot (\delta + \varphi)$, v checks if at least one $tx^{R,u}_{lock}$ has been confirmed on $(v,u) \in \mathcal{D}$, and then v posts all $\left(tx^{R,v}_{lock}, \sigma^{R,v}_{lock}, \sigma^{R,wv}_{lock}\right)$ on $(w,v) \in \mathcal{D}$.
- Otherwise, if not all of the $tx^{L,w}_{lock}$ have been confirmed (for $v = \ell_1$) or none of the $tx^{R,u}_{lock}$ has been confirmed (if $v \neq \ell_1$), after the corresponding timeouts T^L_v, party v computes $sk^{L,u}_{vu} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\left(C^{L,v}_{vu}\right)$ and $sk^L_{vu} = sk^{L,v}_{vu} \oplus sk^{L,u}_{vu}$. v generates a refund transaction $tx^{L,v}_{rfd} = \left(pk^L_{vu}, pk_v, p^L_{vu}\right)$, computes $\sigma^{L,v}_{rfd} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\left(sk^L_{vu}, tx^{L,v}_{rfd}\right)$, and then posts $\left(tx^{L,v}_{rfd}, \sigma^{L,v}_{rfd}\right)$ on its outgoing arcs $(v,u) \in \mathcal{D}$.

Fig. 5: GumSwap Protocol (Part I).

Swap Lock Phase

For each $(v,u) \in \mathcal{D}$, v computes $\sigma_{lock}^{P,v} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_v, tx_{lock}^{P,v}\big)$ under pk_v. According to the role of v in $\mathcal{D}$, v proceeds as follows.

- If $v = \ell_i$, at time $t + 3diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)$, v checks if at least one $tx_{lock}^{R,u}$ has been confirmed on its outgoing arcs $(v,u) \in \mathcal{D}$, and then v posts $\big(tx_{lock}^{P,v}, \sigma_{lock}^{P,v}, \sigma_{lock}^{P,vu}\big)$ on its outgoing arcs $(v,u) \in \mathcal{C}_i$.
- If $v = \ell_i$, $v \neq \ell_1$, at time $t + (3diam(\mathcal{D}_\ell) + 2\mathcal{D}_{\ell_i}(v)) \cdot (\delta + \varphi)$, v checks if all $tx_{lock}^{P,v}$ have been confirmed on both $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$, and then v posts $\big(tx_{lock}^{P,v}, \sigma_{lock}^{P,v}, \sigma_{lock}^{P,vu}\big)$ on $(v,u) \in \mathcal{C}_j$.
- If $v \neq \ell_i$, at the time $t + (3diam(\mathcal{D}_\ell) + 2\mathcal{D}_{\ell_i}(v)) \cdot (\delta + \varphi)$, v checks if all $tx_{lock}^{P,w}$ have been confirmed on its incoming arcs $(w,v) \in \mathcal{D}$, and then v posts $\big(tx_{lock}^{P,v}, \sigma_{lock}^{P,v}, \sigma_{lock}^{P,vu}\big)$ on its outgoing arcs $(v,u) \in \mathcal{D}$.
- Otherwise, if not all of the $tx_{lock}^{R,v}$ have been confirmed (for $v = \ell_i$) or none of the $tx_{lock}^{P,u}$ has been confirmed (for $v \neq \ell_1$), after the corresponding timeouts T_v^R, party v computes $sk_{vu}^{R,w} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C_{wv}^{R,v}\big)$ and $sk_{vu}^{R} = sk_{vu}^{R,v} \oplus sk_{vu}^{R,w}$. v generates a refund transaction $tx_{rfd}^{R,v} = \big(pk_{wv}^R,\ pk_v,\ pk_{wv}^R + p_{wv}^L\big)$, computes $\sigma_{rfd}^{R,v} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{wv}^R, tx_{rfd}^{R,v}\big)$, and then posts $\big(tx_{rfd}^{R,v}, \sigma_{rfd}^{R,v}\big)$ on its incoming arcs $(w,v) \in \mathcal{D}$.

Swap Complete or Timeout Phase

For each $(w,v) \in \mathcal{D}$, according to the role of v in $\mathcal{D}$, party v proceeds as follows.

- If $v = \ell_1$, at time $t + 5diam(\mathcal{D}_\ell) \cdot (\delta + \varphi)$, v checks if all $tx_{lock}^{P,w}$ have been confirmed on its incoming arcs $(w,v) \in \mathcal{D}$, and then v computes $\sigma_{swp}^{wv} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{wv}, y_1\big)$. It posts $\big(tx_{swp}^{wv}, \sigma_{swp}^{wv}\big)$ on each incoming arc $(w,v) \in \mathcal{D}$.
- If $v = \ell_i$, $i \neq 1$, at time $t + 5diam(\mathcal{D}_\ell) \cdot (\delta + \varphi) + \mathcal{D}(v,\ell_1) \cdot (2\delta + \varphi)$, v checks if at least one tx_{swp}^{vu} has been broadcast on $(v,u) \in \mathcal{C}_j$, and then v extracts $y_j \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Ext}\big(\sigma_{swp}^{vu}, \tilde{\sigma}_{swp}^{vu}, Y_j\big)$. v computes $\sigma_{swp}^{wv} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{wv}, y_i\big)$ on $(w,v) \in \mathcal{C}_i$ and $\sigma_{swp}^{wv} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{wv}, y_j\big)$ on $(w,v) \in \mathcal{C}_j$. It posts $\big(tx_{swp}^{wv}, \sigma_{swp}^{wv}\big)$ on each corresponding arc $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$.
- If $v \neq \ell_i$, at time $t + 5diam(\mathcal{D}_\ell) \cdot (\delta + \varphi) + \mathcal{D}(v,\ell_1) \cdot (2\delta + \varphi)$, v checks if at least one tx_{swp}^{vu} has been broadcast on $(v,u) \in \mathcal{D}$, and then v extracts $y_i \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Ext}\big(\sigma_{swp}^{vu}, \tilde{\sigma}_{swp}^{vu}, Y_i\big)$. v computes $\sigma_{swp}^{wv} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{Adapt}\big(\tilde{\sigma}_{swp}^{wv}, y_i\big)$ and posts $\big(tx_{swp}^{wv}, \sigma_{swp}^{wv}\big)$ on each $(w,v) \in \mathcal{D}$.
- Otherwise, if not all of the $tx_{lock}^{P,w}$ have been confirmed (for $v = \ell_1$) or none of the tx_{swp}^{vu} has been broadcast (for $v \neq \ell_1$), after the timeout T_v^P, v computes $sk_{vu}^{P,u} \leftarrow \Sigma_{\mathsf{VTD}}.\mathsf{ForceOp}\big(C_{vu}^{P,u}\big)$ and $sk_{vu}^{P} = sk_{vu}^{P,v} \oplus sk_{vu}^{P,u}$. v generates a refund transaction $tx_{rfd}^{P,v} = \big(pk_{vu}^P,\ pk_v,\ amt_v + p_{vu}^R\big)$, computes $\sigma_{rfd}^{P,v} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}\big(sk_{vu}^P, tx_{rfd}^{P,v}\big)$, and then posts $\big(tx_{rfd}^{P,v}, \sigma_{rfd}^{P,v}\big)$ on each $(v,u) \in \mathcal{D}$.

Fig. 6: GumSwap Protocol (Part II)

and *lock principal* to distinguish two types of asset transfers, although both refer to transferring assets to a shared address.

Remark 1. *We define $\mathcal{D}(v,u)$ as the maximum distance from vertex v to vertex u. We define $diam(\mathcal{D}_\ell)$ as the longest path that ends at the main leader ℓ_1, starting from any non-main leader and allows at most one repeat visit to a non-main leader when transitioning out of its home component. We also define $\mathcal{D}_{\ell_i}(u), u \in \mathcal{C}_i$ as the longest path that starts at some leader ℓ_k (a descendant of ℓ_i) and ends at u, without repeating any vertices except for the possible exception of revisiting ℓ_k once. For example, in Fig. 3 (see Section II-B), the digraph has $diam(\mathcal{D}_\ell) = 5$, corresponding to the path $B - D - E - B - C - A$, and $\mathcal{D}_A(C) = 4$, corresponding to the path $B - D - E - B - C$.*

We illustrate the GumSwap process as follows, and the detailed description is shown in Figs. 5 and 6.

Lock premium setup. Each party v first generates two shared addresses pk_{vu}^L and pk_{vu}^P with u on (v,u), and a shared address pk_{wv}^R with w on (w,v). Party v holds key shares $(sk_{vu}^{L,v}, sk_{wv}^{R,v}, sk_{vu}^{P,v})$, u holds $(sk_{vu}^{L,u}, sk_{vu}^{P,u})$, and w holds $sk_{wv}^{R,w}$. v then constructs transactions $(tx_{lock}^{R,v}, tx_{swp}^{wv})$ on (w,v), and $tx_{lock}^{P,v}$ on (v,u). Each leader $v = \ell_i$ samples a statement witness pair (Y_i, y_i) and broadcasts Y_i to all participants in its home component $\mathcal{C}_i$. All lock transactions that spend from shared addresses are jointly signed, and all swap transactions are jointly pre-signed. Each v also computes three VTD commitments on its key shares and sends the corresponding commitments to u and w.

After all transactions and signatures are verified, each leader $v = \ell_i$ deposits its p_{vu}^L on $(v,u) \in \mathcal{C}_i$. Each non-main leader $v = \ell_i$ with $i \neq 1$ waits until all $tx_{lock}^{L,w}$ on both $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$ have been confirmed, and then deposits p_{vu}^L on $(v,u) \in \mathcal{C}_j$. Each follower v waits until all $tx_{lock}^{L,w}$ on $(w,v) \in \mathcal{D}$ have been confirmed, and then deposit p_{vu}^L on $(v,u) \in \mathcal{D}$.

Redemption premium setup. The main leader $v = \ell_1$ checks if all $tx_{lock}^{L,w}$ have been confirmed on $(w,v) \in \mathcal{D}$, and then v deposits p_{wv}^R on $(w,v) \in \mathcal{D}$. Each non-main leader $v = \ell_i$ with $i \neq 1$ waits until at least one transaction $tx_{lock}^{R,u}$ has been confirmed on $(v,u) \in \mathcal{C}_j$, and then it deposits p_{wv}^R on both $(w,v) \in \mathcal{C}_i$ and $(w,v) \in \mathcal{C}_j$. Each follower v waits until at least one transaction $tx_{lock}^{R,u}$ has been confirmed on $(v,u) \in \mathcal{D}$, and then it deposits p_{wv}^R on $(w,v) \in \mathcal{D}$. Otherwise, after the corresponding timeouts, v reclaims its p_{vu}^L on $(v,u) \in \mathcal{D}$ if p_{vu}^L is still deposited.

Swap lock. Each leader $v = \ell_i$ waits until at least one transaction $tx_{lock}^{R,u}$ has been confirmed on $(v,u) \in \mathcal{C}_i$, and then locks its principal amt_v and refunds p_{vu}^L on $(v,u) \in \mathcal{C}_i$.

// The 2PC protocol $\Pi_{\mathsf{SIG}}^{\mathsf{JKGen}}$

1. Party v computes $(pk_{vu}^v, sk_{vu}^v) \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{KGen}(1^\lambda)$ and sends pk_{vu}^v to u.
2. Party u computes $(pk_{vu}^u, sk_{vu}^u) \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{KGen}(1^\lambda)$ and sends pk_{vu}^u to v.
3. Parties v and u generates the shared address pk_{vu}.

// The 2PC protocol $\Pi_{\mathsf{SIG}}^{\mathsf{Sign}}$

It takes private inputs sk_{vu}^v and sk_{vu}^u held by parties v and u respectively, and public input tx:

1. Set secret key as $sk_{vu} := sk_{vu}^v \oplus sk_{vu}^u$
2. Compute signature $\sigma_{tx} \leftarrow \Sigma_{\mathsf{SIG}}.\mathsf{Sign}(sk_{vu}, tx)$ and sends σ_{tx} to both parties v and u.
3. Parties v and u respectively check if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{vu}, tx, \sigma_{tx}) = 1$, and stop otherwise.

// The 2PC protocol $\Pi_{\mathsf{AS}}^{\mathsf{pSign}}$

It takes private inputs sk_{vu}^v and sk_{vu}^u held by parties v and u respectively, and public inputs (tx, Y):

1. Set secret key as $sk_{vu} := sk_{vu}^v \oplus sk_{vu}^u$
2. Compute signature $\tilde{\sigma}_{tx} \leftarrow \Sigma_{\mathsf{AS}}.\mathsf{pSign}(sk_{vu}, tx, Y)$ and sends $\tilde{\sigma}_{tx}$ to both parties v and u.
3. Parties v and u respectively check if $\Sigma_{\mathsf{AS}}.\mathsf{pVrfy}(pk_{vu}, tx, \sigma_{tx}, Y) = 1$, and stop otherwise.

Fig. 7: Subroutines of 2PC Protocols

Each non-main leader $v = \ell_i$ with $i \neq 1$ waits until all transactions $tx_{lock}^{P,w}$ have been confirmed on both $(w, v) \in \mathcal{C}_i$ and $(w, v) \in \mathcal{C}_j$, and then v locks amt_v and refunds p_{vu}^L on $(v, u) \in \mathcal{C}_j$. Each follower v waits until all transactions $tx_{lock}^{P,w}$ have been confirmed on $(w, v) \in \mathcal{D}$, and then v locks amt_v and refunds p_{vu}^L on $(v, u) \in \mathcal{D}$. Otherwise, after the corresponding timeouts, party v reclaims $p_{wv}^R + p_{wv}^L$ on $(w, v) \in \mathcal{D}$.

Swap complete or timeout. The main leader $v = \ell_1$ waits until all transactions $tx_{lock}^{P,w}$ have been confirmed on $(w, v) \in \mathcal{D}$, and then redeems $amt_w + p_{wv}^R$ on all $(w, v) \in \mathcal{D}$. Each non-main leader $v = \ell_i$ with $i \neq 1$ waits until at least one swap transaction tx_{swp}^{vu} has been broadcast on $(v, u) \in \mathcal{C}_j$, and then redeems $amt_w + p_{wv}^R$ on both $(w, v) \in \mathcal{C}_i$ and $(w, v) \in \mathcal{C}_j$. Each follower v waits until at least one swap transaction tx_{swp}^{vu} has been broadcast on $(v, u) \in \mathcal{D}$, and then redeems $amt_w + p_{wv}^R$ on $(w, v) \in \mathcal{D}$. Otherwise, after the corresponding timeouts, party v reclaims $amt_v + p_{vu}^R$ on $(v, u) \in \mathcal{D}$.

We formalize the security of the protocol in the UC framework. A formal proof of the following theorem is provided in Appendix B.

Theorem 1. *If Σ_{AS} is a secure adaptor signature scheme w.r.t. a secure digital signature scheme Σ_{SIG} and a hard relation $\mathcal{R}$, Σ_{VTD} is a secure verifiable timed dlog scheme, Π_{SIG} and Π_{AS} are UC-secure 2PC protocols, respectively. Then* GumSwap*, UC-realizes the ideal functionality $\mathcal{F}_{\mathsf{GS}}$ with access to the functionalities ($\mathcal{F}_{clock}$, $\mathcal{F}_{smt}$, $\mathcal{F}_B$).*

C. Security Analysis

We prove that GumSwap ensures atomicity and hedge.

Definition 1 (Atomicity and Hedge)**.** GumSwap *satisfies atomicity and hedge if, for any PPT adversary $\mathcal{A}$ corrupting a subset of parties, the following conditions hold:*

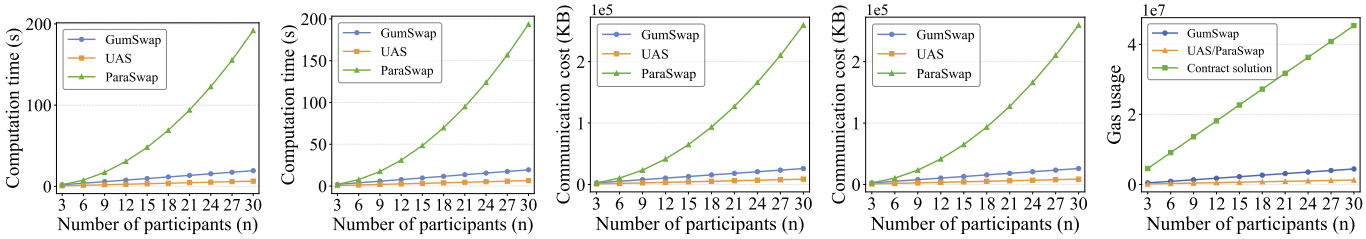
1) *If v deposits its lock premium p_{vu}^L but u fails to deposit its redemption premium p_{vu}^R on (v, u), v must reclaim its p_{vu}^L.*
2) *If v deposits its redemption premium p_{wv}^R but w fails to lock its principal amt_w on (w, v), v must reclaims its locked assets and obtain w's p_{wv}^L as compensation.*
3) *If v locks its principal amt_v that has been redeemed on (v, u), v must obtain w's principal amt_w on all (w, v) and reclaims its locked premiums.*
4) *If v locks its principal amt_v but u fails to redeem amt_v on (v, u), v must reclaims its locked premiums and principal on (v, u), and obtain u's p_{vu}^R as compensation.*

Theorem 2. *If Σ_{AS} is a secure adaptor signature scheme w.r.t. a secure digital signature scheme Σ_{SIG} and a hard relation $\mathcal{R}$, and Σ_{VTD} is a secure verifiable timed dlog scheme, then* GumSwap *achieves atomicity and hedge.*

Proof. Suppose, for contradiction, that GumSwap does not satisfy atomicity and hedge. It implies that at least one of these conditions does not hold.

1) Party v locks its lock premium p_{vu}^L while u fails to lock its p_{vu}^R, and then v fails ito reclaim its p_{vu}^L. This means that (i) v cannot obtain a valid signature on its refund transaction $tx_{rfd}^{L,v}$; (ii) u posts a delayed transaction $tx_{lock}^{R,u}$ after $T_v^L + \delta + \varphi$, which conflicts with v's $tx_{rfd}^{L,v}$ broadcast at time T_v^L, and $tx_{lock}^{R,u}$ is then confirmed. The former case implies that the *soundness* of Σ_{VTD} is broken, and the latter case implies the security of the underlying blockchain is broken.

2) Party v deposits its redemption premium p_{wv}^R while w fails to lock its amt_w, and then v fails to obtain w's p_{wv}^L or to reclaim its assets. It means that (i) v cannot obtain a valid signature on the refund transaction $tx_{rfd}^{R,v}$; (ii) w can obtain a valid signature on its refund transaction $tx_{rfd}^{L,w}$ before T_w^L; or (iii) w posts a delayed transaction $tx_{lock}^{P,w}$ after $T_v^R + \delta + \varphi$, which conflicts with v's $tx_{rfd}^{R,v}$ broadcast at time T_v^R, and then $tx_{lock}^{P,w}$ is confirmed. The case (i) implies that the *soundness* of Σ_{VTD} is broken; the case (ii) implies the *privacy* of Σ_{VTD} is broken; and the case (iii) implies the security of the underlying blockchain is broken.



(a) Computation time (SIG = Schnorr) (b) Computation time (SIG = ECDSA) (c) Communication cost (SIG = Schnorr) (d) Communication cost (SIG = ECDSA) (e) Gas Usage

Fig. 8: Off-chain and on-chain costs of GumSwap, UAS [4], ParaSwap [6], and a contract-based solution [13].

3) Party v locks its principal amt_v that has been redeemed, but v fails to obtain w's amt_w or to reclaim its premiums. It means that (i) v cannot obtain a valid signature on tx_{swp}^{wv}; (ii) w can obtain a valid signature on $tx_{rfd}^{P,w}$ before T_w^P. The former case implies that the *unforgeability*, the *pre-signature adaptability*, or the *witness extractability* of Σ_{AS} is broken, and the latter case implies the *privacy* of Σ_{VTD} is broken.

4) Party v locks its principal amt_v while u fails to redeem amt_v, but v fails to obtain u's premium p_{vu}^R or to reclaim its principal and premiums. It means that (i) v fails to obtain a valid signature on its refund transaction $tx_{rfd}^{P,v}$ after T_v^P; or (ii) u posts a delayed transaction tx_{swp}^{vu} after $T_v^P + \delta$, which conflicts with v's $tx_{rfd}^{P,v}$ broadcast at time T_v^P, and then tx_{swp}^{vu} is confirmed. The former case implies that the *soundness* of Σ_{VTD} is broken, and the latter case implies the security of the underlying blockchain is broken. □

V. Performance Evaluation

We implement prototypes of our GumSwap protocol using C language to demonstrate the feasibility of our constructions and to its performance in terms of computation time, communication overhead, and on-chain costs. All experiments are conducted on a laptop equipped with an Ryzen AI 9 HX 370 2.00 GHz, 32 GB RAM, running Ubuntu. We instantiate Schnorr and ECDSA signatures over the *secp256k1* curve, and instantiate VTD with statistical parameters $n = 64$ and $t = n/2$ under a 1024-bit RSA modulus. We reuse the two-party computation (2PC) protocols of jointly key generation, signing, and pre-signing algorithms from [5]. The source code is available at [29].

For a clear comparison with existing baselines, we conduct experiments on a single-leader swap digraph, where each party is restricted to one incoming arc and one outgoing arc.

Computation time. We measure the computation time of GumSwap and compare it with the UAS [4] and ParaSwap [6] protocols under different numbers of participants. The results are shown in Figs 8a and 8b. It can be observed that (i) the computation time of GumSwap grows approximately linearly with the number of participants, since each party is required to compute pre-specified signatures and commitments with its counterparties on every arc; and (ii) when $n = 30$, GumSwap takes 19.296s in total for Schnorr-based instance and 19.43s for ECDSA-based instance, corresponding to roughly 0.64s per party, which is practical in real-world scenarios, especially considering that swap windows in cryptocurrency exchanges typically range from 6 to 24 hours. Moreover, compared to the UAS baseline [4], GumSwap incurs only a small overhead, i.e., 0.429s per party for Schnorr and 0.431s per party for ECDSA, since GumSwap requires two additional premium lock phases to mitigate griefing attacks. Compared with ParaSwap [6], GumSwap reduces each party's computation time by about $10\times$ when $n = 30$, i.e., to 5.75s for Schnorr and 5.807s for ECDSA, since ParaSwap requires each party to perform n VTD and 2PC operations on each arc.

Communication overhead. We measure the communication overhead based on the amount of messages exchanged between parties, as shown in Figs. 8c and 8d. Specifically, when $n = 30$, GumSwap incurs a total communication cost of 25,942KB for Schnorr and 25,956KB for ECDSA (corresponding to ∼864KB per party), which is acceptable for typical WLANs with a bandwidth of at least 1 Mbps. Compared to UAS (8,647.5 KB for Schnorr and 8,652.2 KB for ECDSA), GumSwap incurs an additional 17,295 KB for Schnorr and 17,304.4 KB for ECDSA due to two additional premium lock phases, increasing by ∼576 KB per party. In contrast, GumSwap achieves about a $10\times$ reduction compared to ParaSwap, saving 233,482.5 KB for Schnorr and 233,609.1 KB for ECDSA in total, i.e., reducing ∼7,782 KB per party.

On-chain costs. We evaluate the gas usage of GumSwap and compare it with the UAS [4], ParaSwap [6], and the contract-based solution [13] using the Foundry tool [30]. The results are summarized in Fig. 8e. Specifically, each party consumes 147,000 gas in GumSwap, which is 105,000 gas more than UAS and ParaSwap (42,000 gas per party), since GumSwap requires each party to deposit two additional premiums. Compared to the contract-based solution [13], GumSwap achieves a $10.3\times$ reduction in gas consumption, saving 1,364,710 gas per party (about 40,941,300 gas in total when $n = 30$), which corresponds to about $2.0 USD per party (about $59.6 USD in total when $n = 30$) based on the ETH/USD exchange rate in Jan. 2026.

VI. Related Work

Thyagarajan et al. [4] introduced the first universal atomic swap protocol by using AS and VTD instead of HTLC scripts. Subsequently, Gökay et al. [31] developed an atomic swap protocol compatible with BLS-based blockchains. Hanzlik et

al. [19] proposed Sweep-UC, which enables users to swap coins with an untrusted intermediary while achieving unlinkability and atomicity. Wang and Xiao [32] proposed Pace, which employs mixers as relays to ensure unlinkability in a non-custodial exchange setting. Liu et al. [33] proposed universal, trustless, large-value payment protocols that introduce a revocation operation for scriptless payment channels. Ni et al. [5] proposed PipeSwap, which addresses timeout race attacks in universal two-party swaps by a two-hop mechanism. Xiao et al. [6] presented universal multi-party swaps that parallelize the lock and redemption steps. However, these universal swap protocols remain vulnerable to griefing attacks.

In finance, optionality refers to the value derived from obtaining the right, without any obligation, to make a future investment. The optionality inherent in atomic swaps, first identified by a user with ID "ZmnSCPxj" in 2018 [34], is the main incentive for griefing attacks. BitMEX Research [10] and Robinson [11] further elaborated that griefing risk is intrinsic, and the optionality cannot be eliminated in atomic swaps. Han et al. [12] conducted a quantitative analysis of this optionality in atomic swaps. Xue and Herlihy [13] introduced a contract-based, griefing-free swap protocol and a premium distribution mechanism on strongly-connected directed graphs that compensates conforming parties with the deviating party's premium. However, this solution is not compatible with scriptless blockchains and cannot support reuniclus graphs.

Nadahalli et al. [17] implemented a griefing-free two-party swap using only HTLC scripts. Singh et al. [35] further achieved a griefing-free and bribery-safe swap requiring just four on-chain transactions. Mazumdar [14] proposed a quick swap that enables a compliant party to claim compensation from a deviating party before the timeout. Zhuo et al. [15] introduced a fast settlement protocol that incentivizes a deviating party to promptly refund the counterparty's locked assets. Kai et al. [16] proposed a credit mechanism that adjusts premium amounts based on participants' reputations. Unfortunately, these solutions consider only two-party swaps and cannot be extended to the multi-party swaps.

Robinson [11] proposed packetized payments, in which a transaction is split into a series of minimal swaps to reduce the impact of griefing, but this method is feasible only for fungible and divisible cryptocurrencies. Some swap protocols assume the presence of a centralized entity that will not engage in griefing attacks. Arwen protocol [18] relied on a centralized exchange incentivized by its revenue and reputation to avoid malicious behavior. Tairi et al. [36] used a registration protocol to ensure that a centralized exchange locks assets only when a user has locked an equal amount.

In the context of multi-party swaps, Herlihy [24] first modeled atomic cross-chain swaps using directed graphs, providing a theoretical framework for multi-party swaps. Belotti et al. [37] proposed a generic game-theoretic framework that formalizes the swap problem to characterize equilibria of swap protocols. Narayanam et al. [38] proposed an augmented HTLC protocol using multi-party computation for the atomic multi-party-and-asset exchanges model. Chan et al. [39] explored the preference of the swap model and presented a general swap model that allows each party to specify their preference. Clark et al. [25] proved that HTLC-based protocols can only achieve atomic swaps on reuniclus graphs.

VII. Conclusion

In this paper, we propose GumSwap, the first griefing-free universal multi-party swap protocol, which requires only basic signature verification from the underlying blockchains. We identify the timeout race attack, the premium escape attack, and the topological limitation in universal multi-party swaps. To address these challenges, we impose minimum timeout intervals and introduce an asset migration mechanism. We also design a premium distribution mechanism tailored to reuniclus graphs. Our experimental results demonstrate that GumSwap is efficient and practical. Moving forward, we plan to develop cryptographic primitives to extend the GumSwap protocol, enabling it to support arbitrary strongly connected digraphs.

References

[1] G. Wang, Q. Wang, and S. Chen, "Exploring blockchains interoperability: A systematic survey," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–38, 2023.

[2] A. Augusto, R. Belchior, M. Correia, A. Vasconcelos, L. Zhang, and T. Hardjono, "Sok: Security and privacy of blockchain interoperability," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 3840–3865.

[3] X. Chen, J. Lai, C. Lin, X. Huang, and D. He, "Cryptographic primitives in script-based and scriptless payment channel networks: A survey," *ACM Computing Surveys*, vol. 57, no. 10, pp. 1–34, 2025.

[4] S. A. Thyagarajan, G. Malavolta, and P. Moreno-Sanchez, "Universal atomic swaps: Secure exchange of coins across all blockchains," in *2022 IEEE symposium on security and privacy (SP)*. IEEE, 2022, pp. 1299–1316.

[5] P. Ni, A. Tian, and J. Xu, " Warning! The Timeout T Cannot Protect You From Losing Coins PipeSwap: Forcing the Timely Release of a Secret for Atomic Cross-Chain Swaps ," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, 2025, pp. 1566–1583.

[6] D. Xiao, C. Zhang, H. Deng, J. Liang, L. Wang, and L. Zhu, "Parallelizing universal atomic swaps for multi-chain cryptocurrency exchanges," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 4073–4092.

[7] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, "Generalized channels from limited blockchain scripts and adaptor signatures," in *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part II 27*. Springer, 2021, pp. 635–664.

[8] S. A. K. Thyagarajan, A. Bhat, G. Malavolta, N. Döttling, A. Kate, and D. Schröder, "Verifiable timed signatures made practical," in *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, 2020, pp. 1733–1750.

[9] T. Nolan, "Alt chains and atomic transfers," https://bitcointalk.org/index.php?topic=193281.0, May 2013, bitcoin Forum.

[10] BitMEX Research, "Atomic swaps and distributed exchanges: The inadvertent call option," https://www.bitmex.com/blog/atomic-swaps-and-distributed-exchanges-the-inadvertent-call-option, 2019.

[11] D. Robinson, "Htlcs considered harmful," 2019. [Online]. Available: http://diyhpl.us/wiki/transcripts/stanford-blockchain-conference/2019/htlcs-considered-harmful/

[12] R. Han, H. Lin, and J. Yu, "On the optionality and fairness of atomic swaps," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, 2019, pp. 62–75.

[13] Y. Xue and M. Herlihy, "Hedging against sore loser attacks in cross-chain transactions," in *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, 2021, pp. 155–164.

[14] S. Mazumdar, "Towards faster settlement in htlc-based cross-chain atomic swaps," in *2022 IEEE 4th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2022, pp. 295–304.
[15] F. Zhuo, Z. Song, L. Jia, H. Zhang, Z. Li, and Y. Sun, "Enabling fast settlement in atomic cross-chain swaps," in *International Conference on Security and Privacy in Communication Systems*. Springer, 2023, pp. 395–412.
[16] K. Wang, D. Wang, H. Zhi, Y. Chen, and X. Zhang, "Hash time lock with dynamic premium based on credit in cross-chain transaction," in *2024 IEEE International Conference on Blockchain (Blockchain)*, 2024, pp. 123–130.
[17] T. Nadahalli, M. Khabbazian, and R. Wattenhofer, "Grief-free atomic swaps," in *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2022, pp. 1–9.
[18] E. Heilman, S. Lipmann, and S. Goldberg, "The arwen trading protocols," in *Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers 24*. Springer, 2020, pp. 156–173.
[19] L. Hanzlik, J. L. Sri, A. Thyagarajan, and B. Wagner, "Sweep-uc: Swapping coins privately," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3822–3839.
[20] M. Christ and J. Bonneau, "Limits on revocable proof systems, with implications for stateless blockchains," in *International Conference on Financial Cryptography and Data Security*. Springer, 2023, pp. 54–71.
[21] P. Ananth, A. Poremba, and V. Vaikuntanathan, "Revocable cryptography from learning with errors," in *Theory of Cryptography Conference*. Springer, 2023, pp. 93–122.
[22] M. R. A. Muid, T. Chung, and T. Hoang, "Accurevoke: Enhancing certificate revocation with distributed cryptographic accumulators," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 664–681.
[23] G. Scopelliti, C. Baumann, F. Alder, E. Truyen, and J. T. Mühlberg, "Efficient and timely revocation of v2x credentials," in *Proceedings of the 2024 Network and Distributed System Security Symposium (NDSS'24)*. Internet Society, 2024.
[24] M. Herlihy, "Atomic cross-chain swaps," in *Proceedings of the 2018 ACM symposium on principles of distributed computing*, 2018, pp. 245–254.
[25] E. Clark, C. Georgiou, K. Poon, and M. Chrobak, "On htlc-based protocols for multi-party cross-chain swaps," in *35th International Symposium on Algorithms and Computation (ISAAC 2024)*, 2024.
[26] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*. IEEE, 2001, pp. 136–145.
[27] R. Canetti, Y. Dodis, R. Pass, and S. Walfish, "Universally composable security with global setup," in *Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007. Proceedings 4*. Springer, 2007, pp. 61–85.
[28] J. Katz, U. Maurer, B. Tackmann, and V. Zikas, "Universally composable synchronous computation," in *Theory of Cryptography Conference*. Springer, 2013, pp. 477–498.
[29] Anonymous, "Gumswap code," Anonymous GitHub, https://anonymous.4open.science/r/GumSwap/.
[30] "Foundry," https://github.com/foundry-rs/foundry, GitHub.
[31] H. Gokay, F. Baldimtsi, and G. Ateniese, "Atomic swaps for boneh–lynn–shacham (bls) based blockchains," in *European Symposium on Research in Computer Security*. Springer, 2024, pp. 341–361.
[32] J. Wang and B. Xiao, "Pace: Privacy-preserving and atomic cross-chain swaps for cryptocurrency exchanges," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, 2025, pp. 985–1002.
[33] Z. Liu, C. Lin, D. He, X. Huang, and L. Pu, "Universal and trustless large-value payments in cryptocurrencies," in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*, 2024, pp. 415–426.
[34] ZmnSCPxj, "An argument for single-asset lightning network," https://diyhpl.us/~bryan/irc/bitcoin/bitcoin-dev/linuxfoundation-pipermail/lightning-dev/2018-December/001752.txt, 2018.
[35] K. Singh, V. J. Ribeiro, and S. Mandal, "4-swap: Achieving grief-free and bribery-safe atomic swaps using four transactions," *arXiv preprint arXiv:2508.04641*, 2025.
[36] E. Tairi, P. Moreno-Sanchez, and M. Maffei, "A 2 l: Anonymous atomic locks for scalability in payment channel hubs," in *2021 IEEE symposium on security and privacy (SP)*. IEEE, 2021, pp. 1834–1851.
[37] M. Belotti, S. Moretti, M. Potop-Butucaru, and S. Secci, "Game theoretical analysis of cross-chain swaps," in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, 2020, pp. 485–495.
[38] K. Narayanam, V. Ramakrishna, D. Vinayagamurthy, and S. Nishad, "Atomic cross-chain exchanges of shared assets," in *Proceedings of the 4th ACM Conference on Advances in Financial Technologies*, 2022, pp. 148–160.
[39] E. Chan, M. Chrobak, and M. Lesani, "Cross-chain swaps with preferences," in *2023 IEEE 36th Computer Security Foundations Symposium (CSF)*, 2023, pp. 261–275.

APPENDIX A
ADDITIONAL PRELIMINARIES

A. Adaptor Signature

Adaptor signatures allow users to generate an encrypted signature (pre-signature) on a message m, which is not valid by itself but can be adapted into a valid signature if the user knows a certain witness. Here, we give a formal description of adaptor signatures, following the definitions and security experiments from [7].

Definition 2 (Adaptor Signature)**.** *An adaptor signature w.r.t. a signature scheme* $\Sigma_{\mathsf{SIG}} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ *and a hard relation* $\mathcal{R}$*, comprising four algorithms* $\Sigma_{\mathsf{AS}} = (\mathsf{pSign}, \mathsf{pVrfy}, \mathsf{Adapt}, \mathsf{Ext})$.

- $\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$*: It is a PPT algorithm that on input a private key* sk*, a message* $m \in \{0,1\}^*$*, and a puzzle* $Y \in L_{\mathcal{R}}$*, outputs a pre-signature* $\tilde{\sigma}$.
- $0/1 \leftarrow \mathsf{pVrfy}(m, pk, Y, \tilde{\sigma})$ *: It is a DPT algorithm that, on input, a message* $m \in \{0,1\}^*$*, a public key* pk*, and a puzzle* $Y \in L_{\mathcal{R}}$*, and a pre-signature* $\tilde{\sigma}$*, outputs a bit* $0/1$.
- $\sigma \leftarrow \mathsf{Adapt}(\tilde{\sigma}, y)$ *: It is a DPT algorithm that, on input, a pre-signature* $\tilde{\sigma}$ *and a witness* y*, outputs a valid signature* σ.
- $y \leftarrow \mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$ *: It is a DPT algorithm that, on input, a signature* σ*, a pre-signature* $\tilde{\sigma}$*, and a puzzle* $Y \in L_{\mathcal{R}}$*, outputs a witness* y *such that* $(Y, y) \in \mathcal{R}$.

Definition 3 (Pre-signature Correctness)**.** *An adaptor signature scheme* Σ_{AS} *satisfies pre-signature correctness stating that for every* $m \in \{0,1\}^*$ *and each* $(Y, y) \in \mathcal{R}$*, the following condition is satisfied:*

$$\Pr\left[\begin{array}{c} \mathsf{pVrfy}(pk, m, Y, \tilde{\sigma}) = 1 \\ \wedge \mathsf{Vrfy}(pk, m, \sigma) = 1 \\ \wedge (Y, y) \in \mathcal{R} \end{array} : \begin{array}{c} (pk, sk) \leftarrow \mathsf{KGen}(1^\lambda) \\ \tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y) \\ \sigma := \mathsf{Adapt}(\tilde{\sigma}, y) \\ y := \mathsf{Ext}(\sigma, \tilde{\sigma}, Y) \end{array}\right] = 1.$$

Definition 4 (aEUF-CMA security)**.** *An adaptor signature scheme is unforgeable if, for any PPT adversary* $\mathcal{A}$*, there exists a negligible function* ν *such that* $\Pr[\mathrm{aSigForge}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda) = 1] \leq \nu(\lambda)$*, where the experiment* $aSigForge_{\mathcal{A},\Sigma_{\mathsf{AS}}}$ *is defined in Figure 9.*

Definition 5 (Pre-signature Adaptability)**.** *An adaptor signature scheme satisfies pre-signature adaptability if for any message* $m \in \{0,1\}^*$*, any statement/witness pair* $(Y, y) \in \mathcal{R}$*, any public key* pk*, and any pre-signature* $\tilde{\sigma} \in \{0,1\}^*$ *with*

$\underline{\mathrm{aSigForge}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda)}$
$\mathcal{Q} := \emptyset$
$(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$
$(Y, y) \leftarrow \mathsf{RGen}(1^\lambda)$
$m \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{S}}(\cdot),\mathcal{O}_{\mathsf{pS}}(\cdot,\cdot)}(pk, Y)$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\sigma \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{S}}(\cdot),\mathcal{O}_{\mathsf{pS}}(\cdot,\cdot)}(\tilde{\sigma})$
return $m \notin \mathcal{Q} \wedge \mathsf{Vrfy}(pk, m, \sigma)$

$\underline{\mathcal{O}_{\mathsf{S}}(m)}$
$\sigma \leftarrow \mathsf{Sign}(sk, m)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return σ
$\underline{\mathcal{O}_{\mathsf{pS}}(m, Y)}$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return $\tilde{\sigma}$

Fig. 9: Unforgeabiltiy Experiment of AS

$\underline{\mathrm{aWitExt}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda)}$
$\mathcal{Q} := \emptyset$
$(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$
$(m, Y) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{S}}(\cdot),\mathcal{O}_{\mathsf{pS}}(\cdot,\cdot)}(pk)$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\sigma \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{S}}(\cdot),\mathcal{O}_{\mathsf{pS}}(\cdot,\cdot)}(\tilde{\sigma})$
$y' := \mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$
return $m \notin \mathcal{Q} \wedge (Y, y') \notin \mathcal{R}$
$\wedge \mathsf{Vrfy}(pk, m, \sigma)$

$\underline{\mathcal{O}_{\mathsf{S}}(m)}$
$\sigma \leftarrow \mathsf{Sign}(sk, m)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return σ
$\underline{\mathcal{O}_{\mathsf{pS}}(m, Y)}$
$\tilde{\sigma} \leftarrow \mathsf{pSign}(sk, m, Y)$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
return $\tilde{\sigma}$

Fig. 10: Witness Extractability Experiment of AS

$\mathsf{pVrfy}(pk, m, Y, \tilde{\sigma}) = 1$, *we have* $\mathsf{Vrfy}(pk, m, \mathsf{Adapt}(\tilde{\sigma}, y)) = 1$.

Definition 6 (Witness Extractability)**.** *An adaptor signature scheme is witness extractable if, for every PPT adversary $\mathcal{A}$, there exists a negligible function* $\Pr[\mathrm{aWitExt}_{\mathcal{A},\Sigma_{\mathsf{AS}}}(\lambda) = 1] \leq \nu(\lambda)$*, where the experiment* $\mathrm{aWitExt}_{\mathcal{A},\Sigma_{\mathsf{AS}}}$ *is defined in Figure 10.*

B. Verified Timed Dlog

A verifiable timed dlog (discrete logarithm) [8], [4] is defined w.r.t. a group $\mathbb{G}$ of order q and a generator g. Here, a committer generates a timed commitment with timing hardness T for a secret $sk \in \mathbb{Z}_q$ such that $pk = g^{sk}$ where pk is public and sk is referred to as the dlog (discrete logarithm) of pk (w.r.t. g). The verifier checks the well-formedness of the timed commitment and is able to learn the value sk by forcibly opening the commitment within time T. The formal definition is as follows.

Definition 7 (Verifiable Timed Dlog)**.** *A verifiable timed dlog is a tuple of four algorithms* $\Sigma_{\mathsf{VTD}} = (\mathsf{Commit}, \mathsf{Verify}, \mathsf{Open}, \mathsf{ForceOp})$ *as follow:*

- $(C, \pi) \leftarrow \mathsf{Commit}(sk, T)$: *the commit algorithm takes a discrete log value $sk \in \mathbb{Z}_q$ (generated using $\Sigma_{\mathsf{SIG}}.\mathsf{KGen}(1^\lambda)$), a hiding time T and outputs a commitment C and a proof π.*
- $0/1 \leftarrow \mathsf{Verify}(pk, C, \pi)$: *the verify algorithm takes a public key pk, a commitment C of hardness T, and a proof π and accepts the proof by outputting 1 if and only if, the value sk embedded in C is a valid commitment. Otherwise, it outputs 0.*
- $(sk, r) \leftarrow \mathsf{Open}(C)$: *the open phase where the committer takes a commitment C and outputs the committed value sk and the randomness r used in generating C.*
- $sk \leftarrow \mathsf{ForceOp}(C)$: *the force open algorithm takes the commitment C and outputs the value sk.*

Definition 8 (Soundness)**.** *A VTD is sound if there is a negligible function $\nu(\cdot)$ such that for all PPT adversaries $\mathcal{A}$ and all $\lambda \in \mathbb{N}$, the probability that the verification fails is bounded by:*

$$\Pr\left[b_1 = 1 \wedge b_2 = 0 \;\middle|\; \begin{array}{l} (pk, C, \pi, T) \leftarrow \mathcal{A}(1^\lambda), \\ sk \leftarrow \mathsf{ForceOp}(C), \\ b_1 := \mathsf{Verify}(pk, C, \pi), \\ b_2 := (pk = g^{sk}) \end{array} \right] \leq \nu(\lambda).$$

Definition 9 (Privacy)**.** *A VTD is private if there exists a PPT simulator $\mathcal{S}$, a negligible function $\nu(\cdot)$, and a polynomial $\tilde{T}$, such that for all polynomials $T > \tilde{T}$, all PRAM algorithms $\mathcal{A}$ running within a time $t < T$, and all $\lambda \in \mathbb{N}$ it holds that*

$$\left| \begin{array}{l} \Pr\left[\mathcal{A}(pk, C, \pi) = 1 \;\middle|\; \begin{array}{l} sk \leftarrow \{0,1\}^\lambda, \\ pk := g^{sk} \\ (C, \pi) \leftarrow \mathsf{Commit}(sk, T) \end{array} \right] \\ - \Pr\left[\mathcal{A}(pk, C, \pi) = 1 \;\middle|\; \begin{array}{l} sk \leftarrow \{0,1\}^\lambda, \\ pk := g^{sk} \\ (C, \pi) \leftarrow \mathsf{Sim}(pk, T) \end{array} \right] \end{array} \right| \leq \nu(\lambda)$$

Appendix B
Proof of Theorem 1

Proof. We now prove that GumSwap UC-realizes the ideal functionality $\mathcal{F}_{\mathsf{GS}}$.

To show the indistinguishability between the ideal world and the real world, we construct a simulator $\mathcal{S}$ to simulate the protocol in the real world while interacting with the ideal functionality $\mathcal{F}_{\mathsf{GS}}$. At the beginning, $\mathcal{S}$ corrupts any subset of participants. We begin with the real world protocol execution, gradually change the simulation in these hybrids and then we argue about the proximity of neighbouring experiments.

Hybrid $\mathcal{H}_0$: It is the same as the execution of GumSwap;

Hybrid $\mathcal{H}_1$: It is the same as the above execution except that the 2PC protocol Π_{SIG} in the premium setup phase to generate signatures is simulated using the 2PC simulators $\mathcal{S}^{2PC}_{\mathsf{SIG}}$ for the corrupted user;

Hybrid $\mathcal{H}_2$: It is the same as the above execution except that the 2PC protocol Π_{AS} in the premium setup phase to generate pre-signatures is simulated using the 2PC simulators $\mathcal{S}^{2PC}_{\mathsf{AS}}$ for the corrupted user;

Hybrid $\mathcal{H}_3$: It is the same as the above execution except that the adversary corrupts any $v \neq \ell_i$ and outputs a valid swap transaction $(tx^{wv}_{swp}, \sigma^{wv}_{swp})$ on arc $(w, v) \in \mathcal{C}_i$ before the simulator initiates swap operation on behalf of ℓ_i, the simulator aborts;

Hybrid $\mathcal{H}_4$: It is the same as the above execution except that the adversary corrupts any $v = \ell_i$ with $i \neq 1$ and outputs a valid swap transaction $(tx^{wv}_{swp}, \sigma^{wv}_{swp})$ on arc $(w, v) \in \mathcal{C}_j$ before the simulator initiates swap operation on behalf of ℓ_j, the simulator aborts;

Hybrid $\mathcal{H}_5$: It is the same as the above execution except that the adversary corrupts a party u on an arc $(v,u) \in \mathcal{C}_i$ with $v \neq \ell_i$ and outputs a valid swap transaction $(tx_{swp}^{vu}, \sigma_{swp}^{vu})$ before the timeout T_v^P. The simulator outputs $(tx_{swp}^{wv}, \sigma_{swp}^{wv})$ on $(w,v) \in \mathcal{C}_i$, and if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{wv}^P, tx_{swp}^{wv}, \sigma_{swp}^{wv}) \neq 1$, the simulator aborts;

Hybrid $\mathcal{H}_6$: It is the same as the above execution except that the adversary corrupts a party u on an arc $(v,u) \in \mathcal{C}_j$ with $v = \ell_i$ and $i \neq 1$, and outputs a valid swap transaction $(tx_{swp}^{vu}, \sigma_{swp}^{vu})$ before the timeout T_v^P. The simulator outputs $(tx_{swp}^{wv}, \sigma_{swp}^{wv})$ on $(w,v) \in \mathcal{C}_j$ and if $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{wv}^P, tx_{swp}^{wv}, \sigma_{swp}^{wv}) \neq 1$, the simulator aborts;

Hybrid $\mathcal{H}_7$: It is the same as the above execution except that the adversarial v outputs a valid refund transaction $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$, $(tx_{rfd}^{R,v}, \sigma_{rfd}^{R,v})$, and $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$ before timeouts T_v^L, T_v^R, and T_v^P, respectively, the simulator aborts;

Hybrid $\mathcal{H}_8$: It is the same as the above execution except that the adversary corrupts the main leader $v = \ell_1$ and the simulator obtains $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$, $(tx_{rfd}^{R,v}, \sigma_{rfd}^{R,v})$, and $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$ after timeouts T_v^L, T_v^R, and T_v^P, respectively. If any $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{vu}^L, tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v}) \neq 1$, $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{vu}^R, tx_{rfd}^{R,v}, \sigma_{rfd}^{R,v}) \neq 1$, or $\Sigma_{\mathsf{SIG}}.\mathsf{Vrfy}(pk_{vu}^P, tx_{rfd}^{P,v}, \sigma_{rfd}^{P,v}) \neq 1$, the simulator aborts;

Simulator $\mathcal{S}$: The simulator $\mathcal{S}$ is defined as the execution in $\mathcal{H}_8$ while interacting with the ideal functionality $\mathcal{F}_{\mathsf{GS}}$. It simulates the view of the adversary and receives messages from the ideal functionality $\mathcal{F}_{\mathsf{GS}}$.

Below we show the indistinguishability between $\mathcal{H}_0$ and $\mathcal{H}_7$.

Hybrid $\mathcal{H}_0 \approx_c \mathcal{H}_1$: The indistinguishability directly follows from the security of 2PC protocol Π_{SIG}. The security of 2PC protocol Π_{SIG} for signature generation guarantees the existence of $\mathcal{S}_{\mathsf{SIG}}^{2PC}$;

Hybrid $\mathcal{H}_1 \approx_c \mathcal{H}_2$: The indistinguishability directly follows from the security of 2PC protocol Π_{AS}. The security of 2PC protocol Π_{AS} for pre-signature generation guarantees the existence of $\mathcal{S}_{\mathsf{AS}}^{2PC}$;

Hybrid $\mathcal{H}_2 \approx_c \mathcal{H}_3$: The only difference between the hybrids is that in $\mathcal{H}_3$ the simulator aborts, if the adversary corrupts $v \neq \ell_i$ and outputs a valid swap transaction $(tx_{swp}^{wv}, \sigma_{swp}^{wv})$ on $(w,v) \in \mathcal{C}_i$ before the simulator initiate a swap on behalf of ℓ_i. With the unforgeability of adaptor signature, the probability of the event triggered in $\mathcal{H}_3$ is negligible;

Hybrid $\mathcal{H}_3 \approx_c \mathcal{H}_4$: The only difference between the hybrids is that in $\mathcal{H}_4$ the simulator aborts, if the adversary corrupts $v = \ell_i$ and outputs a valid swap transaction $(tx_{swp}^{wv}, \sigma_{swp}^{wv})$ on $(w,v) \in \mathcal{C}_j$ before the simulator initiate a swap on behalf of ℓ_j. With the unforgeability of adaptor signature, the probability of the event triggered in $\mathcal{H}_4$ is negligible;

Hybrid $\mathcal{H}_4 \approx_c \mathcal{H}_5$: The only difference between the hybrids is that in $\mathcal{H}_5$ the simulator aborts, if the adversary corrupts u on an arc $(v,u) \in \mathcal{C}_i$ with $v \neq \ell_i$ and outputs a valid swap transaction $(tx_{swp}^{vu}, \sigma_{swp}^{vu})$, while the simulator cannot obtains a valid $(tx_{swp}^{wv}, \sigma_{swp}^{wv})$ on $(w,v) \in \mathcal{C}_i$; With the pre-signature adaptability and witness extractability of adaptor signature, the probability of the event triggered in $\mathcal{H}_5$ is negligible;

Hybrid $\mathcal{H}_5 \approx_c \mathcal{H}_6$: The only difference between the hybrids is that in $\mathcal{H}_6$ the simulator aborts, if the adversary corrupts u on an arc $(v,u) \in \mathcal{C}_i$ with $v = \ell_i$ and $i \neq 1$, and outputs a valid swap transaction $(tx_{swp}^{vu}, \sigma_{swp}^{vu})$, while the simulator cannot obtains a valid $(tx_{swp}^{wv}, \sigma_{swp}^{wv})$ on $(w,v) \in \mathcal{C}_j$; With the pre-signature adaptability and witness extractability of adaptor signature, the probability of the event triggered in $\mathcal{H}_6$ is negligible;

Hybrid $\mathcal{H}_6 \approx_c \mathcal{H}_7$: The only difference among the hybrids is that the simulator aborts, if the adversary outputs any valid refund transaction $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$, $(tx_{rfd}^{R,v}, \sigma_{rfd}^{R,v})$, or $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$ before the corresponding pre-specific timeouts. With the privacy of VTD, the probability of the event triggered is negligible;

Hybrid $\mathcal{H}_7 \approx_c \mathcal{H}_8$: The only difference among the hybrids is that the simulator aborts, if cannot obtain a valid $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$, $(tx_{rfd}^{R,v}, \sigma_{rfd}^{R,v})$, and $(tx_{rfd}^{L,v}, \sigma_{rfd}^{L,v})$ after the corresponding pre-specific timeouts; With the soundness of VTD, the probability of the event triggered is negligible. □

APPENDIX C
IMPOSSIBILITY ON ARBITRARY DIGRAPHS

We now show that universal multi-party swaps using adaptor signatures (AS) and verifiable timed dlog (VTD) are impossible on arbitrary strongly connected digraphs. The protocols are only feasible on *reuniclus graphs*, as formalized below.

Theorem 3. *A swap digraph $\mathcal{D}$ has a universal atomic swap protocol using AS and VTD if and only if $\mathcal{D}$ is a reuniclus graph.*

We prove this theorem by two implications:

- ($\Rightarrow$): If $\mathcal{D}$ supports a universal atomic swap protocol using AS and VTD, then $\mathcal{D}$ must be a reuniclus graph.
- ($\Leftarrow$): If $\mathcal{D}$ is a reuniclus graph, then there exists a universal atomic swap protocol using AS and VTD on $\mathcal{D}$.

Intuitively, we prove the ($\Leftarrow$) implication by illustrating a multi-party swap on a reuniclus graph $\mathcal{D}$, which have been shown in Section IV-B. The ($\Rightarrow$) implication is established via a reduction that shows any universal atomic swap protocol implies the existence of an HTLC-based atomic swap protocol.

We first specify some fundamental properties that universal swap protocols must fulfill. Then, we establish the ($\Rightarrow$) implication in Theorem 3 through a reduction to HTLC-based atomic swap protocols.

A universal swap protocol based on AS and VTD proceeds in discrete steps. Each party v can perform:

1) *Create a statement-witness pair*: A party generates a statement-witness pair $(Y,y) \in \mathcal{R}$. We assume that the party generates at most one statement-witness pair in a swap protocol.
2) *Lock an asset into a shared address with AS and VTD*: For an arc (v,u), v locks an asset into a shared address controlled by v and u. v obtains a VTD with a timeout

T from u, and u obtains a pre-signature based on the statement Y from v.

3) *Redeem or refund the asset*: If u learns the witness y such that $(Y, y) \in \mathcal{R}$, u adapts the pre-signature using y to redeem the asset; otherwise, v can reclaim the asset by opening the VTD after T.

Lemma 1. *Any universal atomic swap protocol* Π *based on AS and VTD can be transformed into an HTLC-based atomic swap protocol* Π_H.

Proof. It is straightforward to observe that the definition of atomicity in Π is equivalent to the definition of atomicity in Π_H. We now construct an HTLC-based protocol Π_H with the same phases and timeouts as Π, by transforming the cryptographic components of Π into their HTLC-based functional interfaces, as formalized in [25]. We define the transformations as follows.

- *Create statement-witness $\Rightarrow$ create hash-preimage.* A statement-witness $(Y, y) \in \mathcal{R}$ created in Π corresponds to generating a hash-preimage pair (h, s) in Π_H such that $h = H(s)$, where H is a one-wayness hash function.
- *Lock in shared address $\Rightarrow$ create contract.* A shared address on (v, u) with statement Y and timeout T corresponds to a contract in Π_H with hashlock h and timelock T.
- *Redeem via witness $\Rightarrow$ Redeem via preimage.* In Π, u redeems iff it learns y with $(Y, y) \in \mathcal{R}$; in Π_H, u redeems iff it presents s with $H(s) = h$. Note that the witness y in Π is extractable (e.g., from the adapted signature), while the preimage s in Π_H is publicly revealed upon redemption.
- *Refund via VTD $\Rightarrow$ refund via timelock.* In Π, v reclaims after T by opening its VTD; in Π_H, v reclaims after T by the timelock script. Pre-timeout reclamation before T is infeasible in both Π and Π_H.

These transformations demonstrate that the foundational properties of protocol Π directly correspond to the properties required for an HTLC-based atomic swap, as described in [25]. Therefore, a protocol Π on $\mathcal{D}$ can be translated to an HTLC-based atomic swap protocol Π_H on the same $\mathcal{D}$. □

Lemma 2. *If $\mathcal{D}$ has a universal atomic swap protocol based on AS and VTD, then $\mathcal{D}$ must be a reuniclus graph.*

Proof. By Lemma 1, any universal atomic swap protocol based on AS and VTD implies the existence of an HTLC-based atomic protocol on $\mathcal{D}$. According to [25, Lemma 13], we know that if $\mathcal{D}$ has an HTLC-based atomic swap protocol, then $\mathcal{D}$ must be a reuniclus graph. Hence, $\mathcal{D}$ is a reuniclus graph. □